PROJECT HANDBOOK · CP-PLATFORM V0.1.0

CP Platform Handbook

A complete explanation of the competitive programming judge: what it does, how it is built, how every operation flows through it, and how it behaves in specific situations, written to take a reader from no background to a working understanding of every file.

≈3,000	65	14	8	3	5
lines of source	project files	HTTP endpoints	database tables	judged languages	containers in compose

How to read this

Every chapter carries a level tag so you can choose your path:

- EVERYONE assumes no programming knowledge. Parts I and IV are mostly written at this level.
- DEVELOPER assumes you can read code and know roughly what a web server and a database are.
- ADVANCED assumes familiarity with Spring, Docker and HTTP, and goes into the reasoning behind design choices and their failure modes.

If you only have twenty minutes, read chapters 1, 2, 4 and 15. If you are preparing to explain the project in an interview, add chapters 13, 21, 22 and 29. Part IV (specific cases) is designed to be dipped into: each case stands on its own.

Everything here was written by reading the full source. Statements about behaviour were checked against the code, and a few were checked by running it: the backend compiles, the frontend builds, and the output-comparison and JSON behaviours described in chapter 15 were tested directly. Where something was not verified by running, the text says so.

PART I

The idea

What the platform is, what it feels like to use, and the vocabulary the rest of the handbook relies on.

EVERYONE

1 What this project is

CP Platform is a website where people practise programming by solving puzzles, and where a computer checks their answers automatically. "CP" stands for *competitive programming*, the sport behind sites such as Codeforces, LeetCode and HackerRank.

A puzzle (called a **problem**) looks like this: "You are given two numbers. Print their sum." The person solving it does not type the answer. They write a small **program** that can compute the answer for *any* pair of numbers, and submit that program to the website.

The website then acts as an examiner. It holds a set of secret test questions for the problem, for example "3 and 4 → 7", "−10 and 10 → 0", "1,000,000 and 1 → 1,000,001". It runs the submitted program on each question, compares what the program printed with the correct answer, and gives a verdict: `ACCEPTED` if every answer was right, or a specific failure such as `WRONG_ANSWER` or `TIME_LIMIT_EXCEEDED`.

The difficult part: running strangers' code safely

Running a program written by an unknown person on your own server is dangerous. The program could delete files, read secrets, attack other computers over the network, or simply loop forever and use up the machine. An online judge must run that code anyway, thousands of times a day.

This project solves that the same way a laboratory handles a dangerous sample: inside a sealed box. Each run happens inside a **Docker container**, a lightweight isolated environment with no network connection, a fixed amount of memory, a cap on how many processes it may start, a file system it cannot modify (apart from one scratch folder), and a stopwatch. When the run ends, the box is thrown away. Chapter 22 examines each of those walls in detail.

What it is made of, in one paragraph

The part you see in the browser (the **frontend**) is a React web application. It talks to a **backend** server written in Java with the Spring Boot framework. The backend stores users, problems and results in a **PostgreSQL** database, and when a program needs judging it asks **Docker** to start a sealed

container for it. Everything can be started with a single command using Docker Compose.

What it is not (yet)

- It is not a contest system: there are no timed rounds, no penalties, no per-contest rankings.
- It is a single server. One backend process does everything, including judging. Chapter 30 describes how it would be split up to handle more users.
- It has no automated test suite. There is no `src/test` folder in the backend.

EVERYONE

2 A tour from the user's chair

This chapter walks through the site the way a visitor experiences it. Later chapters explain what happens behind each click.

A visitor who is not logged in

The home page lists problems in a table: title, a coloured difficulty badge (Easy, Medium, Hard) and tags such as "arrays" or "math". A dropdown filters by difficulty. The **Leaderboard** link shows users ranked by how many different problems they have solved. Clicking a problem opens its page: the statement, any constraints, and a few *sample tests* (example inputs with their correct outputs) so the solver can see the expected format. Below that sits a code editor, the same editor that powers Visual Studio Code, with a language selector for C++, Python and Java.

A visitor can read everything and even type code, but pressing **Submit** sends them to the login page.

Registering and logging in

The Register page asks for an email, a username (3 to 30 characters) and a password (at least 8 characters). After registering, the visitor is logged in immediately and their username appears in the top bar with a Logout button. Logging in later uses email and password.

Solving a problem

1. The user picks a language. The editor fills with a starter template, for example a C++ `main` function.
2. They write a solution that reads the input from standard input (the keyboard, as far as the program knows) and prints the answer.
3. They press Submit. The page immediately moves to a **submission page** with a grey `PENDING` badge.
4. Every 1.5 seconds the page asks the server for news. After a few seconds the badge turns into the verdict. A score shows what percentage of the tests passed, the runtime of the slowest test is

shown, and a table lists each test as "Sample n" or "Hidden n" with its own verdict.

5. If the code failed to compile, the compiler's error message is shown so the user can fix it.

Solving a problem for the first time adds one to the user's solved count, which moves them up the leaderboard.

The administrator's view

Users with the **ADMIN** role see extra controls. A "+ New Problem" link in the top bar opens a form for title, difficulty, statement, constraints, time limit, memory limit, tags and any number of test cases, each with input, expected output and a "Public (sample)" checkbox. On a problem's page, admins also see **Edit** and **Delete** buttons.

There is no screen for making someone an admin. The first administrator is created by changing their role directly in the database (chapter 25 shows the command).

EVERYONE

3 Words you'll meet

Verdicts

VERDICT	MEANING	PRODUCED BY THIS PROJECT?
`PENDING`	Received, waiting to be judged.	Yes
`RUNNING`	Being judged right now.	Written, but never visible to the user (see 23.3)
`ACCEPTED`	Every test produced the correct output.	Yes
`WRONG_ANSWER`	The program ran and finished, but printed something different from the expected output.	Yes
`TIME_LIMIT_EXCEEDED`	The program did not finish in time.	Yes
`RUNTIME_ERROR`	The program crashed or exited with a non-zero status.	Yes
`COMPILE_ERROR`	The C++ or Java source could not be compiled.	Yes
`MEMORY_LIMIT_EXCEEDED`	The program used too much memory.	No. Defined, never assigned. Memory exhaustion shows up as RUNTIME_ERROR.
`INTERNAL_ERROR`	The judge itself failed.	Yes

Other terms

TERM	MEANING HERE
Problem	A puzzle with a statement, limits, tags and test cases.
Test case	One input together with its correct output. *Public* test cases are shown on the problem page as samples; *hidden* ones are never sent to normal users.
Submission	One attempt: a user's source code for one problem in one language, plus its verdict.
Slug	The URL-friendly name of a problem, e.g. `two-sum` in `/problems/two-sum`.
Container	An isolated process environment started by Docker. Shares the host's kernel but sees its own file system, network and limits.
Image	The template a container is started from. This project uses three runner images: `judge-runner-cpp`, `judge-runner-python`, `judge-runner-java`.
JWT	JSON Web Token. A signed piece of text the server gives you at login, which you send back with every request to prove who you are.
Access token / refresh token	Two JWTs. The access token proves identity for 15 minutes; the refresh token lasts 7 days and is only used to get a new access token.
Role	`USER` or `ADMIN`. Decides what you may do.
Endpoint	One URL plus HTTP method that the backend answers, such as `POST /api/submissions`.
DTO	Data Transfer Object: a small class describing the JSON shape of a request or response.
Entity	A Java class mapped to a database table.
Standard input / output	The text stream a program reads from and the one it prints to. The judge feeds each test's input on standard input and captures standard output.
Exit code	A number a program returns when it ends. 0 means success; anything else is treated as a crash.

PART II

The structure

The pieces, the files, the layers inside the backend, the database tables and every endpoint.

DEVELOPER

4 The moving parts

When the system runs under Docker Compose, five containers start, and during judging more short-lived containers come and go.

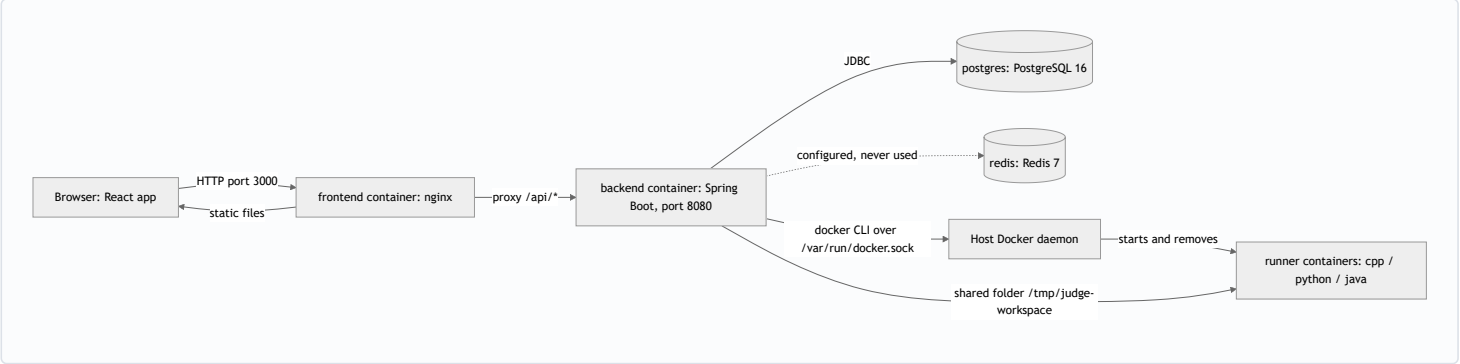


Figure 4.1 · Runtime architecture under Docker Compose.

PIECE	WHAT IT DOES	LIVES IN
Browser app	All screens. Holds the login tokens in the browser's `localStorage`. Calls the backend with JSON over HTTP.	`frontend/src`
nginx (frontend container)	Serves the built JavaScript and HTML. Forwards any `/api/...` request to the backend so the browser only ever talks to one address.	`frontend/nginx.conf`, `frontend/Dockerfile`
Backend	Authentication, problem management, submissions, judging, leaderboard. One Java process.	`backend/src/main/java/com/cpplatform`
PostgreSQL	Stores everything permanent. Tables are created automatically by Hibernate on startup.	`docker-compose.yml` (volume `pgdata`)
Redis	Started and configured, but no code reads or writes it. Reserved for a future leaderboard.	`docker-compose.yml`, `application.yml`
Docker daemon	The host's Docker engine. The backend drives it by running the `docker` command-line tool, which talks to the daemon through the mounted socket file.	Your machine
Runner containers	One container per compile step and one per test case. Each lives for a fraction of a second to a few seconds.	`docker/runners/Dockerfile.*`

Why the backend runs Docker from inside Docker

The backend itself runs in a container, yet it has to start other containers. Rather than run a full Docker engine inside the backend container ("Docker-in-Docker"), the project mounts the host's Docker control socket, `/var/run/docker.sock`, into the backend. The backend container only has the Docker *client*; every `docker run` it issues is executed by the host's daemon. The new runner containers are therefore *siblings* of the backend, not children. This pattern is sometimes called "Docker-out-of-Docker". It is simple and fast, and it has a serious security cost covered in chapter 22.

It also creates a path puzzle. The backend writes each submission's source code to a folder, then asks the daemon to mount that folder into a runner container. The daemon interprets the path *on the host*, not inside the backend container. That is why `docker-compose.yml` mounts `/tmp/judge-workspace` at the identical path on both sides: the folder the backend writes into and the folder the daemon mounts are the same physical directory. If the two paths differed, the daemon would mount an empty folder and every submission would fail to compile.

DEVELOPER

5 Technology stack

Versions below are the ones Maven and npm actually resolved when the project was built for this handbook.

LAYER	TECHNOLOGY	VERSION	ROLE
Backend language	Java	17 (target)	Records and switch expressions are used.
Framework	Spring Boot	3.3.2	Web server (embedded Tomcat), dependency injection, configuration.
Security	Spring Security	6.3.1	Filter chain, URL rules, method rules, password hashing.
Persistence	Spring Data JPA + Hibernate	Hibernate 6.5.2	Maps entities to tables, generates queries from method names.
Database driver	PostgreSQL JDBC	managed by Boot	Connection to Postgres.
JSON	Jackson	2.17.2	Converts DTO records to and from JSON.
Tokens	JJWT	0.12.6	Creates and verifies JWTs.
Boilerplate	Lombok	1.18.34	Generates getters, setters, builders, constructors and the logger.
Validation	Jakarta Bean Validation	managed by Boot	`@NotBlank`, `@Email`, `@Size` on request DTOs.
Database	PostgreSQL	16 (alpine image)	All persistent data.

LAYER	TECHNOLOGY	VERSION	ROLE
Cache (unused)	Redis	7 (alpine image)	Configured via `spring-boot-starter-data-redis`, not used.
Frontend library	React	18.3	UI components.
Routing	React Router	6.x	Client-side pages.
HTTP client	Axios	1.x	API calls, token interceptors.
Code editor	@monaco-editor/react	4.6	VS Code's editor in the browser. Loads Monaco itself from a CDN at runtime.
Build tool	Vite	5.4	Dev server with API proxy; production build to `dist/`.
Web server	nginx	alpine image	Serves the built frontend and proxies the API.
Sandbox	Docker	host engine	Runs every compile and test in isolation.
Runner toolchains	GCC 13, Python 3.11, Temurin JDK 17	from runner images	Compile and run submissions.

DEVELOPER

6 The repository, file by file

Every file in the project, with what it is responsible for. Line counts are approximate.

```
cp-platform/
├── README.md                          overview, run instructions, deferred features
├── docker-compose.yml                 5 services: postgres, redis, backend, frontend (+ volume)
├── hs_err_pid31500.log                unrelated crash log from VS Code's Java tooling; safe to delete
├── docker/runners/
│   ├── Dockerfile.cpp                 gcc:13 + non-root user 10001, workdir /box
│   ├── Dockerfile.python              python:3.11-slim + same user
│   ├── Dockerfile.java                eclipse-temurin:17-jdk + same user
│   └── README.md                      how to build the runner images
├── backend/
│   ├── Dockerfile                     2-stage: Maven build on JDK 17, then JRE 17 + docker CLI
│   ├── pom.xml                        dependencies (Spring Boot 3.3.2 parent)
│   └── src/main/
│       ├── resources/application.yml   all configuration, env-var overridable
│       └── java/com/cpplatform/
│           ├── CpPlatformApplication.java   entry point; @EnableAsync
│           ├── config/
│           │   └── SecurityConfig.java      filter chain, URL rules, CORS, BCrypt, auth manager
│           ├── security/
│           │   ├── JwtService.java          create / parse / validate tokens
│           │   ├── JwtAuthFilter.java       reads Bearer header, sets the logged-in user
│           │   ├── RestAuthEntryPoint.java  JSON 401 responses
│           │   └── RestAccessDeniedHandler.java JSON 403 responses
│           ├── controller/                  HTTP endpoints (thin)
│           │   ├── AuthController.java      /api/auth/register, login, refresh
│           │   ├── ProblemController.java   /api/problems CRUD + admin view
│           │   ├── SubmissionController.java /api/submissions submit, get, mine
│           │   ├── LeaderboardController.java /api/leaderboard
│           │   └── UserController.java      /api/users/me
│           ├── service/                     business logic
│           │   ├── AuthService.java         register, login, refresh, token issuing
│           │   ├── UserDetailsServiceImpl.java loads users for password checking
│           │   ├── ProblemService.java      create, list, view, edit, delete, slugs, tags
│           │   ├── SubmissionService.java   store submission, start judging, views
│           │   ├── JudgeService.java        the judge: workspace, docker runs, verdicts
│           │   ├── RunnerConfig.java        per-language image, file name, commands
│           │   └── LeaderboardService.java  solved-count ranking
│           ├── repository/                  7 Spring Data interfaces (queries)
│           ├── entity/                      6 tables as classes + 3 enums
│           │   ├── User, Problem, Tag, TestCase, Submission,
│           │   │   SubmissionResult, UserProgress
│           │   └── Role, Language, SubmissionStatus
│           ├── dto/                         JSON shapes as Java records
│           │   ├── AuthDtos, ProblemDtos, SubmissionDtos, LeaderboardDtos
│           └── exception/
│               ├── ApiException.java        error with an HTTP status
│               └── GlobalExceptionHandler.java turns exceptions into JSON
└── frontend/
    ├── Dockerfile                     node 20 build, then nginx
    ├── nginx.conf                     static files + /api proxy + SPA fallback
    ├── package.json, vite.config.js, index.html
    └── src/
        ├── main.jsx                   mounts the app: Router → AuthProvider → App
        ├── App.jsx                    top bar, routes, RequireAuth / RequireAdmin
        ├── styles.css                 dark theme, verdict badges
        ├── api/client.js              axios instance, token attach + refresh on 401
        ├── context/AuthContext.jsx    who is logged in; login/register/logout
        └── pages/
            ├── ProblemList.jsx, ProblemDetail.jsx
```

```
├── SubmissionStatus.jsx, Leaderboard.jsx
├── Login.jsx, Register.jsx
└── AdminNewProblem.jsx, AdminEditProblem.jsx
```

The README refers to an `architecture.md` "shared earlier"; that file is not in the repository.

DEVELOPER

7 Inside the backend

The backend follows the conventional Spring layering. Each layer only calls the one below it, with one exception noted at the end.

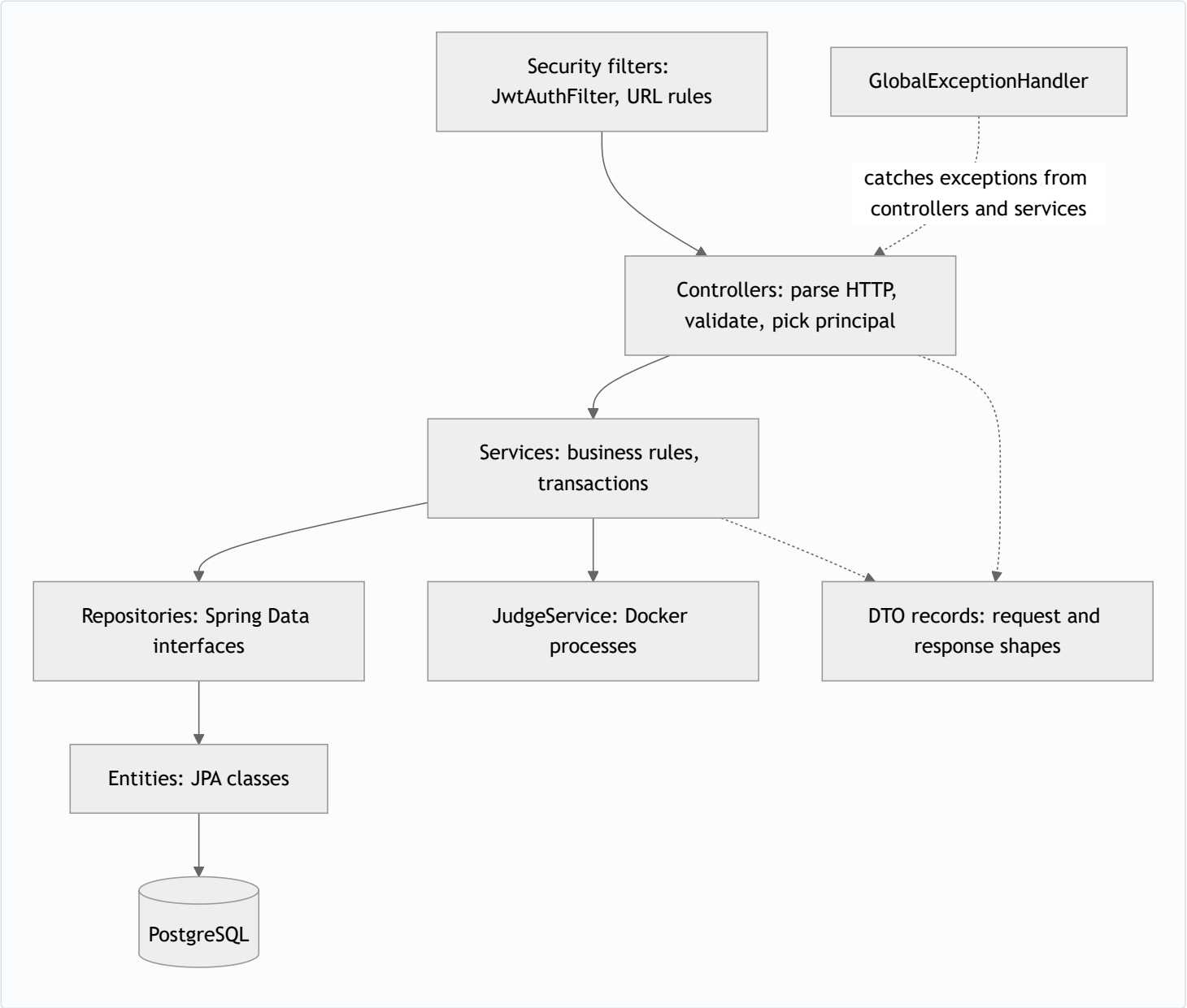


Figure 7.1 · Backend layers.

Controllers

Controllers are deliberately thin. They declare the URL, read path variables and query parameters, trigger validation with `@Valid`, receive the logged-in user via `@AuthenticationPrincipal User`, call one service method and wrap the result in `ResponseEntity.ok(...)`. The principal is the actual `User` entity, because `JwtAuthFilter` places the entity itself into the security context (chapter 13).

Services

All decisions live here: does this email already exist, what is the slug, may this user see this submission, which verdict does this run deserve. Methods that touch several rows are annotated `@Transactional` so they succeed or fail as a whole. `ProblemService.create`, `update`, `delete`, `getBySlug`, `getForAdmin` and `JudgeService.judgeAsync` are transactional; the rest rely on the per-call transactions of repository methods.

Repositories

Seven interfaces extend `JpaRepository`. Spring Data generates their implementations at startup by parsing method names. For example `findByProblemIdAndIsPublicTrue(Long problemId)` becomes "select test cases whose problem id equals ? and whose isPublic is true". Only one query is hand-written: `TestCaseRepository.deleteByProblemId`, a bulk JPQL delete marked `@Modifying`.

Entities

Seven classes map to tables (chapter 8). All use Lombok's `@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder`; fields with default values use `@Builder.Default` so the builder respects them. Timestamps are filled by `@PrePersist` callbacks. Relationships to other entities are `LAZY`: the related row is loaded only when first accessed.

DTOs

Requests and responses are Java *records*, grouped as nested types inside holder classes (`AuthDtos.RegisterRequest`, `ProblemDtos.ProblemDetail`, and so on). Entities are never returned directly. This is what keeps hidden test cases and password hashes out of responses: a DTO can only expose the fields it declares.

Cross-cutting pieces

- `ApiException` carries a message and an HTTP status. Services throw it for expected failures (not found, conflict, forbidden).
- `GlobalExceptionHandler` turns `ApiException` into `{"error": message}` with its status, validation failures into a 400 with one entry per field, and *any other exception* into a 500 with `"Internal server error: " + message`. That last rule matters for several cases in Part IV.

The one layering exception

`ProblemController.delete` calls `SubmissionRepository.countByProblemId` directly and passes the count into the service. It works, but the check belongs inside `ProblemService.delete`.

DEVELOPER

8 The data model

There are no migration scripts. With `spring.jpa.hibernate.ddl-auto: update`, Hibernate inspects the entities at every startup and creates any missing tables and columns. It never drops or renames anything.

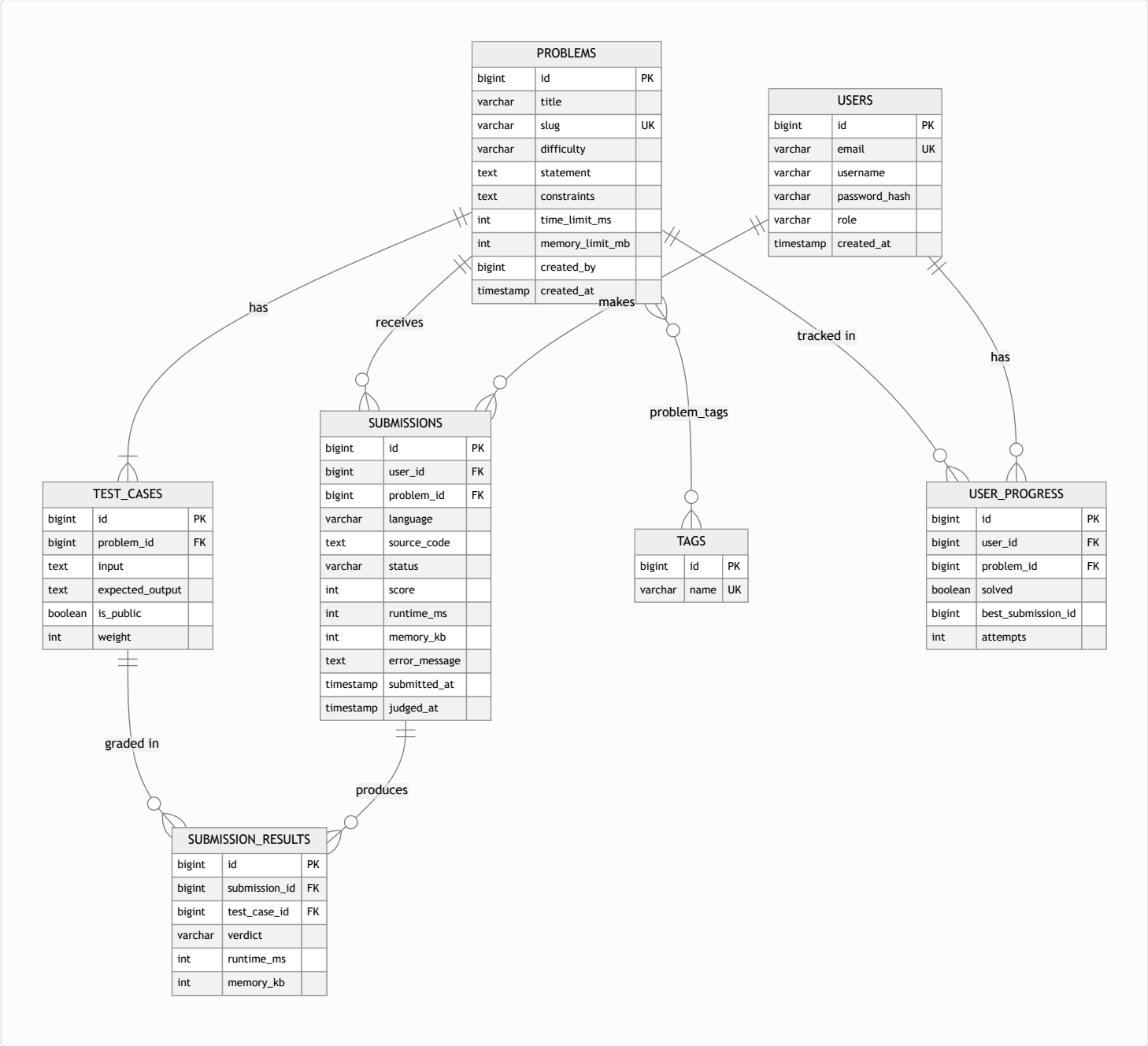


Figure 8.1 · Tables and relationships. `problem_tags` is a join table with columns `problem_id` and `tag_id`.

Table notes

TABLE	IMPORTANT DETAILS
`users`	Email is unique; **username is not**, so two people can both be "alex". Email uniqueness is case-sensitive. `password_hash` is a BCrypt string. `role` is stored as text: `USER` or `ADMIN`.
`problems`	Slug is unique and never changes after creation. `created_by` stores the admin's user id as a plain number, without a foreign key. `memory_limit_mb` is stored and displayed but the judge ignores it.
`tags`, `problem_tags`	Tags are shared between problems and created on first use. Removing a tag from every problem leaves the tag row behind.
`test_cases`	`weight` is stored (default 1) but scoring ignores it; every test counts equally. Mapped from the problem with `cascade = ALL, orphanRemoval = true`, although the service code manages test cases through the repository instead of that collection.
`submissions`	Keeps the full source code. `memory_kb` is never filled in. `runtime_ms` is the slowest test's wall-clock time including container start-up.
`submission_results`	One row per test case per judged submission. Its foreign key to `test_cases` is what prevents editing a problem after anyone has submitted (case C4).
`user_progress`	One row per (user, problem), enforced by a unique constraint. `solved` flips to true on the first accepted submission and stays true. `attempts` counts judged submissions that reached the test stage. `best_submission_id` is the first accepted submission, not the fastest.

How rows come into existence

- **Register** inserts one `users` row.
- **Create problem** inserts one `problems` row, any new `tags`, the `problem_tags` links and one `test_cases` row per test.
- **Submit** inserts one `submissions` row (status PENDING).
- **Judging** updates that submission, inserts one `submission_results` row per test, and inserts or updates one `user_progress` row.

DEVELOPER

9 The API

All endpoints live under `/api` and speak JSON. Authenticated calls send `Authorization: Bearer <accessToken>`.

METHOD	PATH	WHO MAY CALL	PURPOSE
POST	`/api/auth/register`	Anyone	Create account, returns tokens
POST	`/api/auth/login`	Anyone	Log in, returns tokens
POST	`/api/auth/refresh`	Anyone with a refresh token	New token pair
GET	`/api/problems`	Anyone	Paged list; `?difficulty=&page=&size=`
GET	`/api/problems/{slug}`	Anyone	Problem with public tests only

METHOD	PATH	WHO MAY CALL	PURPOSE
GET	`/api/problems/{id}/admin`	ADMIN	Problem with all tests, for editing
POST	`/api/problems`	ADMIN	Create problem
PUT	`/api/problems/{id}`	ADMIN	Replace problem and its tests
DELETE	`/api/problems/{id}`	ADMIN	Delete if no submissions exist
POST	`/api/submissions`	Logged in	Submit code; judging starts in background
GET	`/api/submissions/{id}`	Owner or ADMIN	Status, score and per-test results
GET	`/api/submissions/me`	Logged in	Own submissions, newest first (not used by the UI)
GET	`/api/leaderboard`	Anyone	Top users by solved count; `?limit=` (default 50)
GET	`/api/users/me`	Logged in	Own id, email, username, role (not used by the UI)

Request and response shapes

Register / login / refresh

```
// POST /api/auth/register
{ "email": "ada@example.com", "username": "ada", "password": "correct-horse" }
// POST /api/auth/login
{ "email": "ada@example.com", "password": "correct-horse" }
// POST /api/auth/refresh
{ "refreshToken": "eyJhbGciOi..." }

// Response to all three (AuthResponse)
{
  "accessToken":  "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "userId": 7,
  "username": "ada",
  "role": "USER"
}
```

Create or edit a problem

```
// POST /api/problems   or   PUT /api/problems/{id}
{
  "title": "Sum of Two",
  "difficulty": "EASY",                // EASY | MEDIUM | HARD, required
  "statement": "Read a and b, print a+b.",  // required
  "constraints": "-10^9 ⩽ a, b ⩽ 10^9",   // optional
  "timeLimitMs": 2000,                 // optional, default 2000
  "memoryLimitMb": 256,                // optional, default 256 (not enforced)
  "tags": ["math", "implementation"],
  "testCases": [                       // at least one, checked in the service
    { "input": "3 4", "expectedOutput": "7", "isPublic": true,  "weight": 1 },
    { "input": "-5 5", "expectedOutput": "0", "isPublic": false, "weight": 1 }
  ]
}
// Response (ProblemSummary)
{ "id": 12, "title": "Sum of Two", "slug": "sum-of-two", "difficulty": "EASY", "tags": ["math","implementation"] }
```

View a problem (public)

```
// GET /api/problems/sum-of-two  (ProblemDetail)
{
  "id": 12, "title": "Sum of Two", "slug": "sum-of-two", "difficulty": "EASY",
  "statement": "...", "constraints": "...", "timeLimitMs": 2000, "memoryLimitMb": 256,
  "tags": ["math", "implementation"],
  "publicTestCases": [ { "input": "3 4", "expectedOutput": "7" } ]
}
```

The admin variant (`GET /api/problems/12/admin`) returns the same fields but replaces `publicTestCases` with `testCases` , each including `id` , `isPublic` and `weight` . The list endpoint returns a Spring Data page object: `{"content": [ProblemSummary...], "totalElements": ..., "totalPages": ..., "number": ..., ...}` .

Submit and poll

```
// POST /api/submissions
{ "problemId": 12, "language": "CPP", "sourceCode": "#include ..." }   // CPP | PYTHON | JAVA

// Response, and later GET /api/submissions/31  (SubmissionView)
{
  "id": 31, "status": "WRONG_ANSWER", "score": 50,
  "runtimeMs": 812, "memoryKb": null, "errorMessage": null,
  "submittedAt": "2026-10-09T10:15:02.114Z",
  "results": [
    { "testCaseId": 40, "isPublic": true,  "verdict": "ACCEPTED",     "runtimeMs": 790, "memoryKb": null },
    { "testCaseId": 41, "isPublic": false, "verdict": "WRONG_ANSWER", "runtimeMs": 812, "memoryKb": null }
  ]
}
```

The submit response always has an empty `results` list. The `/me` list also returns empty `results` for each item, to keep it cheap.

Error shapes

STATUS	BODY	WHEN
400	`{"username": "size must be between 3 and 30"}`	Bean validation failed. One key per bad field, **no** `error` key.
400	`{"error": "At least one test case is required"}`	Service-level validation.
401	`{"error": "Authentication required or token expired"}`	No valid access token on a protected endpoint.
403	`{"error": "You don't have permission to do that"}`	Logged in, wrong role (URL or method rule).
403	`{"error": "Not authorized to view this submission"}`	Someone else's submission.
404	`{"error": "Problem not found"}`	Unknown slug or id; unknown submission.
409	`{"error": "Email already registered"}`	Duplicate registration; also deleting a problem with submissions.
500	`{"error": "Internal server error: ..."}`	Anything else, including wrong passwords, malformed JSON and unknown URLs (chapter 24).

DEVELOPER

10 Inside the frontend

The frontend is a single-page application: the browser downloads one HTML file and one JavaScript bundle (about 250 KB, 82 KB compressed), and from then on React swaps screens without reloading the page.

Component tree

```
main.jsx
└── <BrowserRouter>              URL ⇔ screen
    └── <AuthProvider>           who is logged in
        └── <App>                top bar + <Routes>
            ├── /                          ProblemList
            ├── /problems/:slug            ProblemDetail
            ├── /submissions/:id           RequireAuth → SubmissionStatus
            ├── /leaderboard               Leaderboard
            ├── /admin/new-problem         RequireAdmin → AdminNewProblem
            ├── /admin/edit-problem/:id    RequireAdmin → AdminEditProblem
            ├── /login                     Login
            └── /register                  Register
```

There is no catch-all route, so an unknown URL shows the top bar with an empty page beneath it.

Authentication state

`AuthContext` keeps one object, `user = {id, username, role}`, and exposes `login`, `register` and `logout`. On page load it restores `user` from `localStorage`. After a successful login or register it stores three keys:

`accessToken`, `refreshToken` and `user`. Logout calls `localStorage.clear()` and sets `user` to null; nothing is sent to the server.

`RequireAuth` redirects to `/login` if there is no user. `RequireAdmin` redirects to `/` unless `user.role === 'ADMIN'`. These guards are about user experience only. The real enforcement is on the server; anyone can edit `localStorage` to claim to be an admin, and the server will still refuse their admin calls.

The API client

`api/client.js` creates an Axios instance with base URL `/api` and two interceptors:

- **Request interceptor:** reads `accessToken` from `localStorage` and adds the `Authorization` header.
- **Response interceptor:** if a response is 401 and the request has not been retried, it posts the refresh token to `/api/auth/refresh` (with plain Axios so the interceptor does not loop), stores the new pair, and replays the original request. If refreshing fails, it clears storage and does a full-page redirect to `/login`.

The relative base URL is what makes the same build work in development and production. In development, Vite's dev server proxies `/api` to `localhost:8080`. In production, nginx proxies `/api/` to `backend:8080`.

Pages in brief

PAGE	CALLS	NOTABLE BEHAVIOUR
ProblemList	`GET /problems`	Re-fetches when the difficulty filter changes. Shows only page 0 (20 problems); there are no paging controls.
ProblemDetail	`GET /problems/{slug}`, `POST /submissions`, `DELETE /problems/{id}`	Monaco editor with per-language templates. Changing language replaces the code with the template. Delete asks for confirmation with the browser's `confirm()`.
SubmissionStatus	`GET /submissions/{id}` every 1.5 s	Stops polling on any final verdict or on error. Labels each row "Sample n" or "Hidden n" by its position.
Leaderboard	`GET /leaderboard`	Rank, username, solved count.
Login, Register	`POST /auth/login`, `/auth/register`	Show `error` from the response, or a generic message when the response has none.
AdminNewProblem	`POST /problems`	Starts with two empty test cases. Requires at least one test and at least one marked public (the backend only requires one test).
AdminEditProblem	`GET /problems/{id}/admin`, `PUT /problems/{id}`	Loads everything including hidden tests, sends a full replacement.

The production container

The frontend Dockerfile has two stages. A Node 20 image runs `npm install` and `npm run build`; an nginx image receives only the resulting `dist/`

folder and `nginx.conf`. The nginx configuration does three things: serves files from `/usr/share/nginx/html`, proxies `/api/` to the backend with the usual forwarding headers, and answers any other unknown path with `index.html` (`try_files $uri $uri/ /index.html`), so refreshing the browser on `/problems/two-sum` loads the app instead of a 404.

PART III

The flows

Step-by-step accounts of every operation, from system start-up to the moment a verdict appears on screen.

DEVELOPER

11 Starting the system

Compose start-up order

1. **Images build.** `docker compose up --build` builds the backend image (Maven compiles inside a JDK 17 image, then the JAR is copied into a JRE 17 image that also gets the `docker.io` client package) and the frontend image. The three runner images are *not* part of compose and must be built beforehand with the `docker build` commands in the README.
2. **Postgres and Redis start.** Postgres creates the `cpplatform` database and user on first run, storing data in the `pgdata` volume. Compose polls `pg_isready` every 5 seconds, up to 5 tries.
3. **Backend starts** only once Postgres reports healthy (`depends_on: condition: service_healthy`).
4. **Frontend starts** after the backend container has been *created*. Compose does not wait for the backend to be ready, so for the first few seconds API calls through nginx return 502 errors.

What Spring Boot does on start-up

1. Reads `application.yml` , substituting environment variables such as `DB_HOST` and `JWT_SECRET` .
2. Opens a HikariCP connection pool to Postgres (10 connections by default).
3. Hibernate compares the entities with the database and creates or alters tables.
4. Spring Data generates repository implementations from the interfaces.
5. Builds the security filter chain from `SecurityConfig` .
6. `@EnableAsync` activates background execution using Boot's default task executor (8 core threads, unbounded queue).
7. Configures a Redis connection factory. It connects lazily, so start-up succeeds even if Redis is down, and since nothing uses it, it never connects.

8 Embedded Tomcat starts listening on port 8080.

The database starts empty: no users, no problems, no admin.

DEVELOPER

12 Life of an HTTP request

Every API call, whatever it does, passes through the same pipeline.
Understanding it once explains most of the behaviour in Part IV.

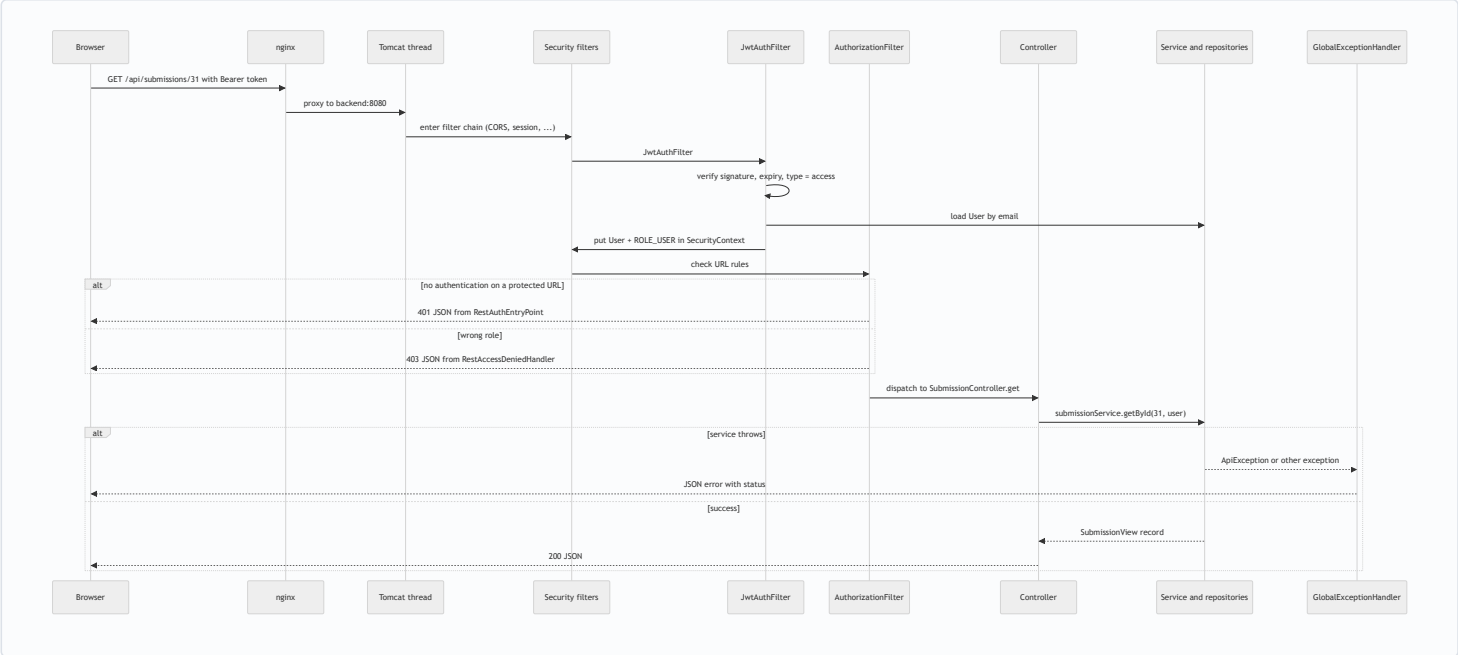


Figure 12.1 · One authenticated request.

Stage by stage

1 **nginx** (production) or the **Vite proxy** (development) forwards the request to the backend.

2 **Tomcat** assigns a worker thread. Everything up to the response happens on that thread.

3 **CORS filter.** Only relevant when the page comes from a different origin. Allowed origins are `http://localhost:*`. Under compose the browser talks to one origin, so CORS never applies.

4 **JwtAuthFilter.** If there is no `Bearer` header, it does nothing. Otherwise it parses the token. Parsing verifies the signature and throws if the token has expired. If the token is an access token and the email in it belongs to a real user, it loads that user from the database and stores an authentication object whose principal is the `User` entity and whose authority is `ROLE_USER` or `ROLE_ADMIN`. Any exception is swallowed, leaving the request unauthenticated.

5 **Authorization.** URL rules are checked in order; the first match wins. If the rule needs an authenticated user and there is none, the entry point returns 401. If the user lacks the role, the access-denied handler returns 403.

6. **DispatcherServlet** finds the controller method, converts JSON to the request record and runs bean validation if `@Valid` is present.
7. **Method security.** `@PreAuthorize("hasRole('ADMIN')")` on `getForAdmin` and `delete` is evaluated before the method runs (a second check on top of the URL rule).
8. **Controller → service → repository.** Business logic runs; transactional methods open a database transaction.
9. **Serialization.** Jackson writes the returned record as JSON.
10. **Errors.** Exceptions thrown from a controller or service go to `GlobalExceptionHandler`. Exceptions inside the security filters never reach it; they are handled by the entry point and access-denied handler.

The URL rules, in evaluation order

#	RULE	EFFECT
1	`/api/auth/**`	permit all
2	`GET /api/problems/*/admin`	ADMIN only. Must come before rule 3, which would otherwise make it public.
3	`GET /api/problems/**`	permit all
4	`GET /api/leaderboard/**`	permit all (also matches `/api/leaderboard` itself)
5–7	`POST`, `PUT`, `DELETE /api/problems/**`	ADMIN only
8	anything else	must be authenticated

DEVELOPER

13 Accounts and tokens

13.1 Registering

1. The Register page posts `{email, username, password}` via `AuthContext.register`.
2. `/api/auth/**` is public, so no token is needed.
3. Validation: email must be well-formed and non-blank, username 3–30 characters, password at least 8. Failures return 400 with per-field messages.
4. `AuthService.register` checks `existsByEmail`; if taken, 409 "Email already registered".
5. The password is hashed with BCrypt (cost factor 10, random salt built into the hash) and the user is saved with role `USER`.
6. `issueTokens` creates an access token and a refresh token and returns them with the user's id, username and role.

7 The frontend stores the tokens and user in `localStorage`, updates `user` state, and navigates home. The top bar now shows the username.

13.2 Logging in

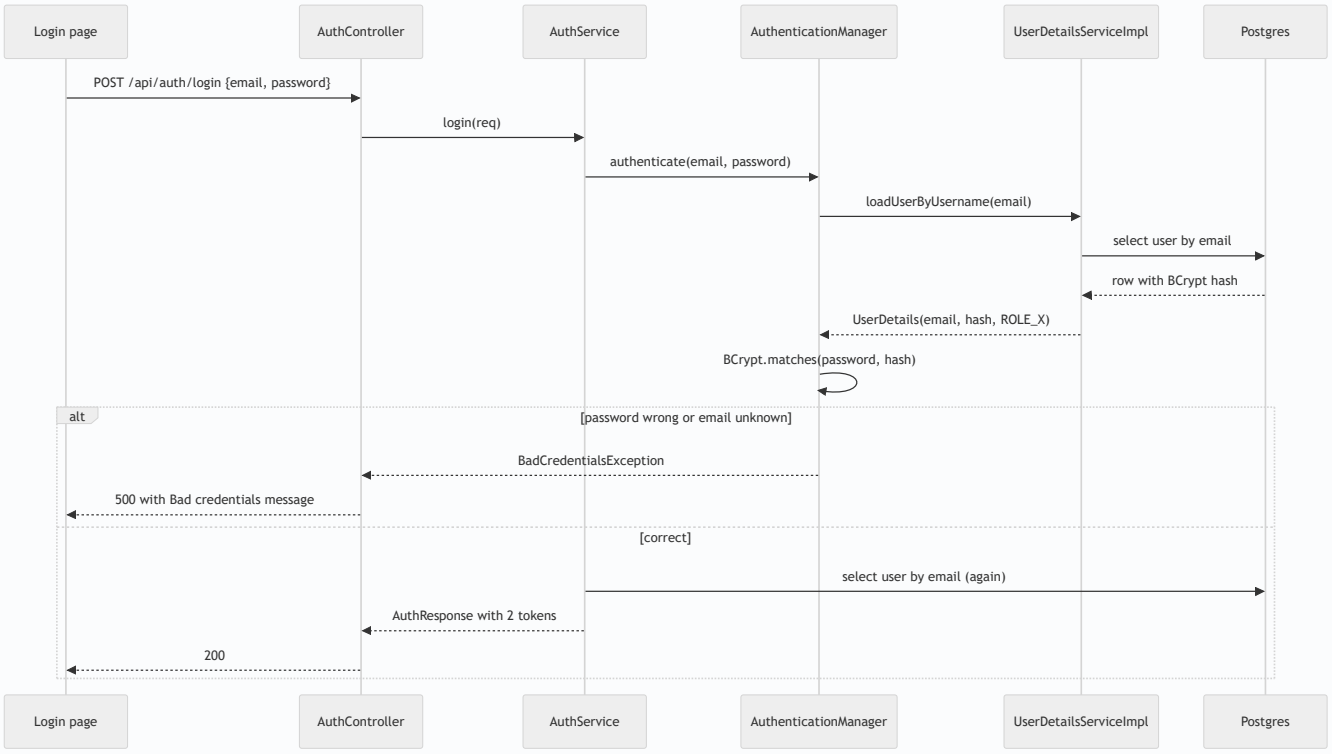


Figure 13.1 · Login. Spring's `DaoAuthenticationProvider` reports an unknown email and a wrong password the same way, so the response does not reveal which accounts exist.

The wrong-password path returns status 500 because `BadCredentialsException` is not handled specifically and falls into the catch-all handler. The login page displays the text "Internal server error: Bad credentials". It should be a 401.

13.3 What is inside the tokens

Both tokens are JWTs signed with HMAC using the `jwt.secret` setting. JJWT chooses the strongest HMAC variant the key length allows; the shipped secrets are 57 and 62 bytes long, which selects HS384. A decoded access token payload looks like this:

```
{
  "sub": "ada@example.com",   // identity: the email
  "role": "USER",
  "userId": 7,
  "type": "access",           // "refresh" in a refresh token
  "iat": 1791540902,          // issued at (seconds)
  "exp": 1791541802           // +15 min; refresh tokens: +7 days
}
```

The refresh token contains only `sub`, `type`, `iat` and `exp`. The `role` and `userId` claims in the access token are never read by the backend; the filter always loads the user from the database. That means a role change takes effect on the user's very next request, at the cost of one database query per authenticated request.

Tokens are signed, not encrypted. Anyone holding one can read its contents; they just cannot change them without breaking the signature.

13.4 An authenticated request

Covered step by step in chapter 12. In short: header → signature and expiry check → type must be "access" → user loaded by email → principal set → URL rule check → controller receives the `User` entity through `@AuthenticationPrincipal`.

13.5 Silent refresh

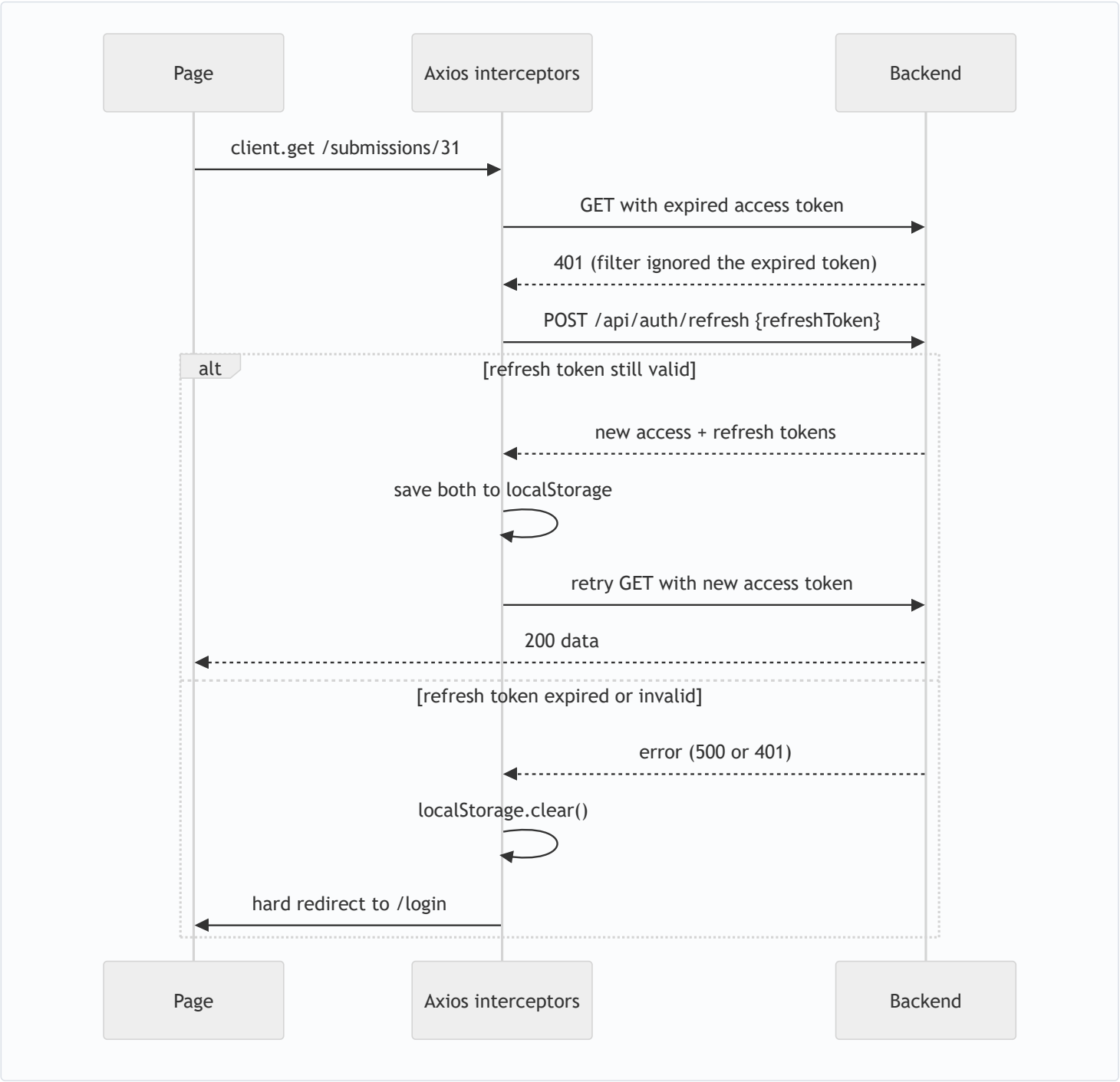


Figure 13.2 · Refresh on 401.

On the server, `AuthService.refresh` checks the token's type is "refresh", finds the user, and issues a brand-new pair. An *expired* refresh token throws inside JJWT before the expiry check in the service is reached, so the response is a 500 rather than the intended 401. The client treats any failure the same way, so the user experience is unaffected.

The old refresh token is not invalidated when a new one is issued. Every refresh token stays valid for its full 7 days.

13.6 Logging out

Logout is purely client-side: `localStorage.clear()` and `user = null`. The server keeps no session and has no token blacklist, so a copied token keeps working until it expires: up to 15 minutes for an access token and up to 7 days for a refresh token.

DEVELOPER

14 Managing problems

14.1 Creating a problem

1. The admin fills the form. The page checks there is at least one test case and at least one public one, then posts the JSON shown in chapter 9.
2. Security: URL rule 5 requires ADMIN. A normal user gets 403 before any code runs.
3. Validation: title, difficulty and statement are required; every test case needs a non-blank input and expected output.
4. `ProblemService.create` opens a transaction and computes the slug (below). If the slug is taken, it appends `-` plus the current time in milliseconds modulo 10000, e.g. `two-sum-4821`.
5. The problem is built with defaults of 2000 ms and 256 MB when limits are absent, and `createdBy` set to the admin's id.
6. Each tag name is looked up and created if missing.
7. The problem is saved, then each test case is saved with a reference to it. If the list is empty, an `ApiException` (400) is thrown and the whole transaction rolls back, including the problem row.
8. The response is a `ProblemSummary`; the page navigates to `/problems/{slug}`.

How slugs are made

`slugify` decomposes accented letters (é → e + accent mark), removes every non-ASCII character, lowercases, deletes anything that is not a letter, digit, whitespace or hyphen, trims, and turns runs of whitespace into single hyphens.

TITLE	SLUG
Two Sum	`two-sum`
Café Orders!	`cafe-orders`
A + B	`a-b`
Two Sum (second time)	`two-sum-second-time`
Two Sum (exact duplicate)	`two-sum-4821` (number varies)

TITLE	SLUG
数组求和	empty string, see case C2

14.2 Viewing a problem

`GET /api/problems/{slug}` is public. `getBySlug` loads the problem, fetches only test cases with `is_public = true`, and builds a `ProblemDetail`. Hidden tests are not just hidden in the UI; they never leave the server on this endpoint, because the DTO has no field for them.

14.3 Listing problems

`GET /api/problems` takes optional `difficulty`, `page` (default 0) and `size` (default 20). With a difficulty it uses `findByDifficulty`, otherwise `findAll`; both are paged. Each problem is mapped to a summary, which reads its tags. Tags are lazily loaded, so this happens after the repository call returns; it works because Spring Boot keeps the database session open for the whole request ("open session in view", chapter 23).

14.4 Editing a problem

1. The edit page loads `GET /api/problems/{id}/admin`. It is protected twice: URL rule 2 and `@PreAuthorize`.
2. The admin changes fields and saves; the page sends `PUT /api/problems/{id}` with the complete problem.
3. `update` overwrites title, difficulty, statement and constraints; it changes limits only if provided; it replaces the tag set; it leaves the slug unchanged so existing links keep working.
4. It then deletes *all* test cases for the problem with one bulk query and inserts the submitted ones as new rows with new ids.
5. The page navigates back to the problem.

> **LIMITATION** Step 4 fails once anyone has submitted to the problem, because their `submission_results` rows still reference the old test-case ids. Postgres rejects the delete, the transaction rolls back, and the admin sees a 500 error. See case C4.

14.5 Deleting a problem

The controller counts submissions for the problem. If there are any, the service throws 409 with an explanatory message. Otherwise it bulk-deletes the test cases, then deletes the problem; Hibernate removes the `problem_tags` links. Tag rows themselves remain.

DEVELOPER ADVANCED

15 Submitting and judging

This is the heart of the project. The code lives in `SubmissionService`, `JudgeService` and `RunnerConfig`.

15.1 The whole journey

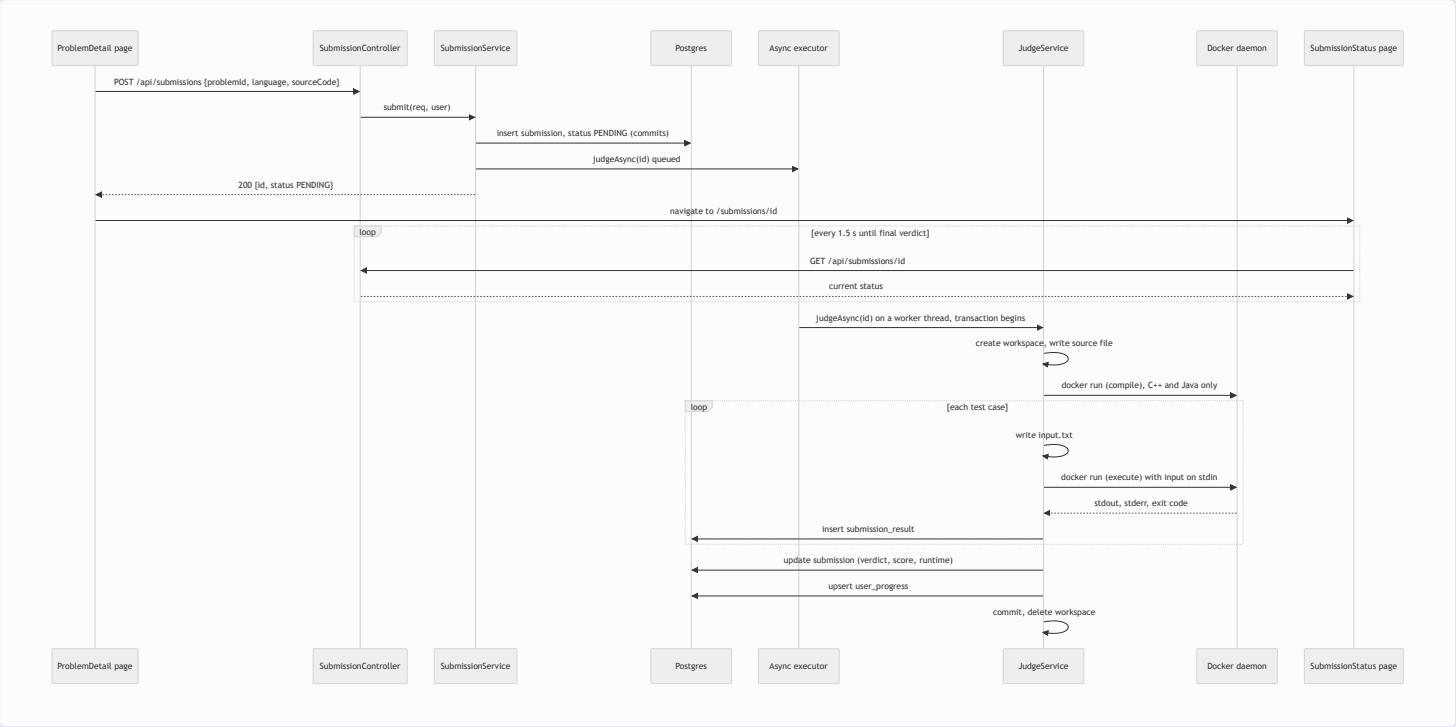


Figure 15.1 · From the Submit button to a stored verdict. The two halves (polling and judging) run at the same time.

15.2 The submit request

1. ProblemDetail checks there is a logged-in user (otherwise it navigates to `/login`) and posts `problemId`, `language` and the editor contents.
2. The URL falls under "anything else", so authentication is required.
3. Validation: `problemId` and `language` must be present and `sourceCode` non-blank. An unknown language string such as `"RUST"` fails JSON conversion and returns a 500.
4. `SubmissionService.submit` loads the problem (404 if missing), builds a `Submission` with status PENDING and saves it. The method is **not** transactional, so the save commits immediately. That is essential: the background worker reads the row from a different thread and connection, and would not find it if it were still uncommitted (see 23.2).
5. It calls `judgeService.judgeAsync(id)`. Because `JudgeService` is a separate Spring bean, the call goes through a proxy which hands the work to the task executor and returns at once.
6. The response is the PENDING submission with empty results. The frontend navigates to `/submissions/{id}`.

15.3 Status lifecycle

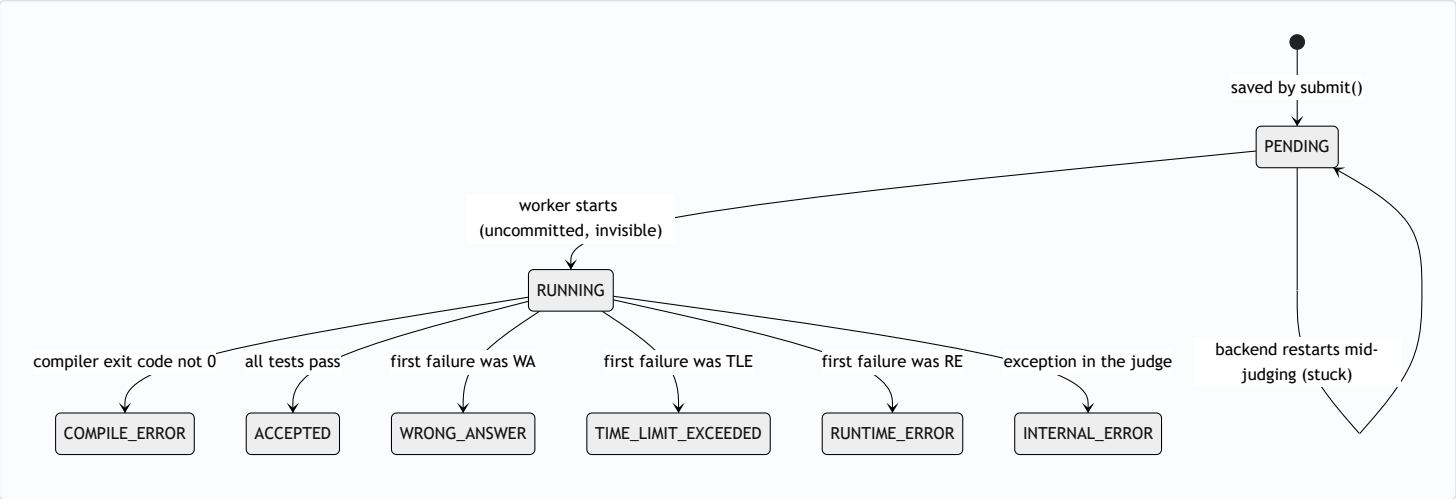


Figure 15.2 · Submission status. MEMORY_LIMIT_EXCEEDED is never produced.

Because the whole of `judgeAsync` runs in one database transaction, the RUNNING status is written but not committed until judging ends, by which time it has been overwritten with the verdict. Pollers therefore see PENDING followed directly by the final verdict.

15.4 Preparing the workspace

The worker creates `{judge.workspace-dir}/sub-{id}`, which is `/tmp/judge-workspace/sub-31` under compose. It sets the folder's permissions to `rwxrwxrwx`, because the runner containers run as user 10001 while the backend runs as root, and the compiler must be able to write its output file into the folder. On a non-POSIX file system (Windows) that call is skipped.

`RunnerConfig.forLanguage` supplies the per-language details:

LANGUAGE	IMAGE	SOURCE FILE	COMPILE COMMAND	RUN COMMAND
CPP	`judge-runner-cpp`	`solution.cpp`	`g++ -O2 -o sol solution.cpp`	`./sol`
PYTHON	`judge-runner-python`	`solution.py`	none	`python3 solution.py`
JAVA	`judge-runner-java`	`Main.java`	`javac Main.java`	`java -cp . Main`

The source code is written into the folder as UTF-8. Note the C++ flags: no `-std=` flag (GCC 13 defaults to C++17) and no extra warnings.

15.5 Anatomy of one sandbox run

Every compile and every test uses the same `docker run` template, assembled in `runDocker`:

```
docker run --rm -i \
  --network=none \
  --memory=256m --memory-swap=256m \
  --cpus=1 \
  --pids-limit=64 \
  --read-only \
  --tmpfs /tmp:rw,size=32m \
  -v /tmp/judge-workspace/sub-31:/box:rw \
  -w /box \
  judge-runner-cpp \
  ./sol
```

FLAG	EFFECT
`--rm`	Delete the container when its process exits.
`-i`	Keep standard input open, so the judge can pipe the test input in.
`--network=none`	No network interface except loopback.
`--memory=256m --memory-swap=256m`	256 MB RAM and no extra swap. Exceeding it gets the process killed by the kernel.
`--cpus=1`	At most one CPU core's worth of time.
`--pids-limit=64`	At most 64 processes and threads. Stops fork bombs.
`--read-only`	The container's own file system cannot be modified.
`--tmpfs /tmp:rw,size=32m`	A 32 MB in-memory scratch `/tmp`, needed by compilers and the JVM.
`-v ...:/box:rw`	The submission's workspace, writable, mounted at `/box`.
`-w /box`	Start in `/box`, so `./sol` and `solution.py` resolve.

The Java process starts `docker` with `ProcessBuilder`. For a test run it copies `input.txt` into the process's standard input and closes it; for a compile it closes standard input immediately. It then waits up to the timeout, and after the process exits it reads all of standard output and standard error. The elapsed time is measured around the wait, so it includes Docker's container start-up and tear-down: typically several hundred milliseconds even for a program that does nothing.

15.6 The compile step

For C++ and Java, one container runs the compile command with a fixed 20-second timeout. If the exit code is not 0 (including a timeout, which is reported as exit code −1), the submission becomes `COMPILE_ERROR`, the first 4,000 characters of the compiler's error output are saved as `errorMessage`, `judgedAt` is set, and the method returns. No test results are written and `user_progress` is not touched, so compile errors do not count as attempts. The compiled `sol` or `Main.class` stays in the workspace for the test runs.

15.7 Running each test

The test cases are loaded with `findByProblemId` (hidden and public together, in database order, normally by id). For each one:

1. The test's input is written to `input.txt`, overwriting the previous one.
2. The timeout is computed as `ceil(max(1, timeLimitMs / 1000)) + 2` seconds.
3. The run command executes in a fresh container with the input piped in.
4. The verdict is decided (figure 15.3).
5. A `submission_results` row is saved with the verdict and wall-clock time.
6. If this is the first non-accepted test, its verdict becomes the overall verdict.

All tests run even after a failure. That gives a complete per-test table and an accurate score, at the cost of judging time, which matters most when every test times out (case A4).

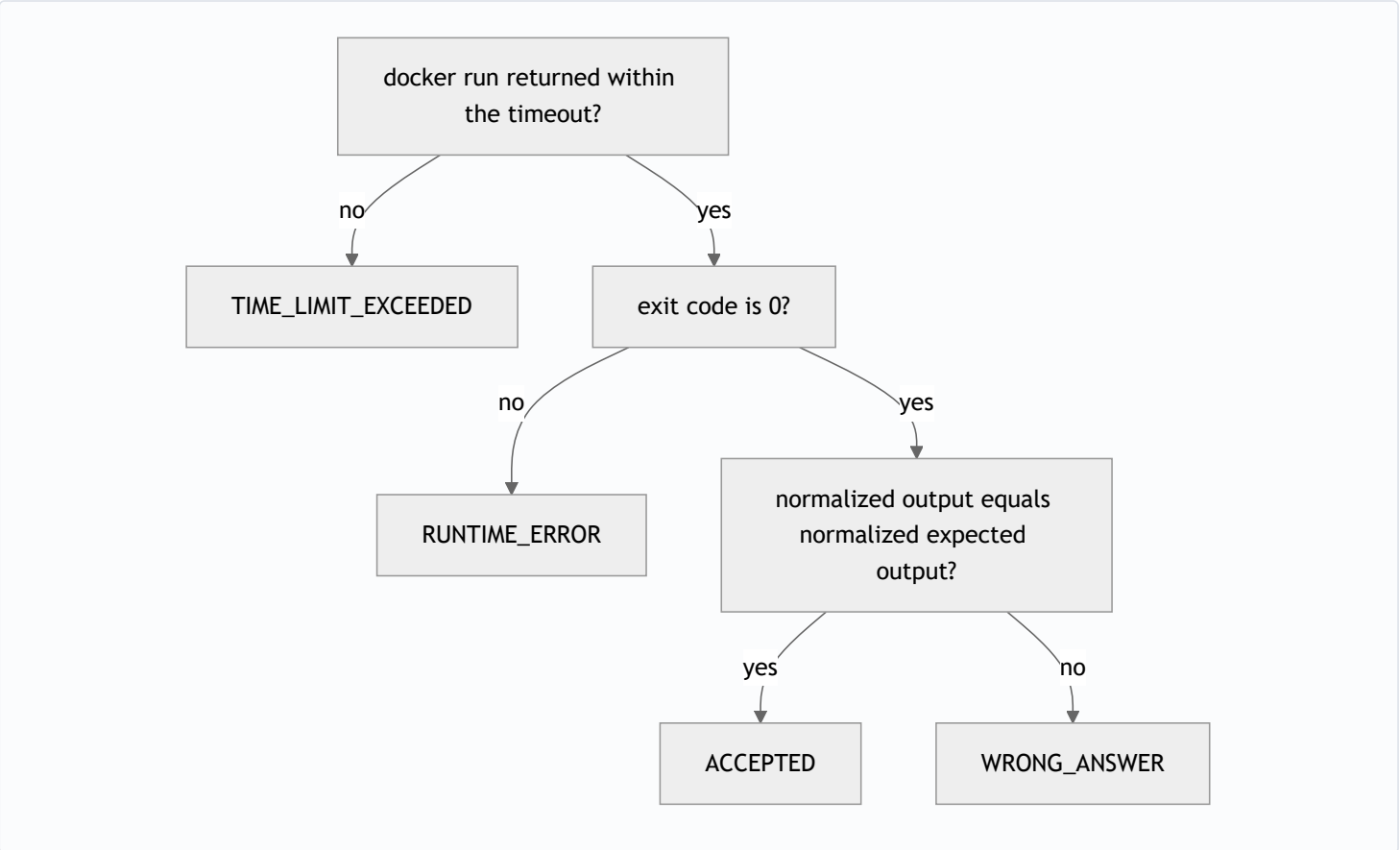


Figure 15.3 · Per-test verdict.

Effective time limits

The timeout applies to the whole `docker run`, container start-up included, and adds two seconds of slack. The limit a solution actually faces is therefore much looser than the number on the problem page:

PROBLEM'S TIME LIMIT	JUDGE'S WALL-CLOCK TIMEOUT
500 ms	3 s

PROBLEM'S TIME LIMIT	JUDGE'S WALL-CLOCK TIMEOUT
1,000 ms	3 s
2,000 ms (default)	4 s
2,500 ms	5 s
5,000 ms	7 s

15.8 Comparing output

Both the program's output and the expected output pass through the same `normalize` function before an exact string comparison:

1. `strip()` : remove all leading and trailing whitespace of the whole text, including blank lines.
2. Remove spaces and tabs that sit immediately before a `\n` .
3. Replace Windows line endings `\r\n` with `\n` .

PROGRAM PRINTED	EXPECTED	RESULT
`7\n`	`7`	`match`
`\n\n7 \n\n`	`7`	`match`
`1 2 3 \n4 5 6 \n`	`1 2 3\n4 5 6`	`match`
`1\r\n2\r\n`	`1\n2`	`match`
`abc \r\ndef`	`abc\ndef`	`no match` (tested: step 2 runs before step 3)
`1 2` (two spaces)	`1 2`	`no match`
`0.30000000000000004`	`0.3`	`no match` (no tolerance for decimals)
`YES`	`yes`	`no match` (case-sensitive)

Problems that accept several correct answers (any valid path, any order) cannot be judged correctly, because there is no custom checker mechanism.

15.9 Finishing up

1. Score = `round(100 × passed ÷ total)` . Two of three tests passing gives 67. Test weights are ignored.
2. The submission gets its final status, score, the maximum runtime across tests, and `judgedAt` .
3. `updateProgress` finds or creates the user's `user_progress` row for this problem, adds one to `attempts` , and, if the verdict is ACCEPTED and the problem was not solved before, sets `solved = true` and records this submission's id.
4. The transaction commits. At this instant the verdict, every result row and the progress update become visible together.

5. The `finally` block deletes the workspace folder, whatever happened.

If any exception escapes during judging (for example the `docker` program does not exist), the catch block marks the submission `INTERNAL_ERROR` with the exception message and saves it.

15.10 Showing the result

`SubmissionStatus` fetches the submission once immediately and then every 1.5 seconds. `getById` returns 403 unless the requester owns the submission or is an admin. When the status is one of the seven final values, polling stops. The page shows the verdict badge, the score, the runtime, the compiler message (if any) and the per-test table. Only compile errors and internal errors carry a message; a program's own error output (for example a Python traceback) is written to the server log and never shown to the user.

DEVELOPER

16 The leaderboard

1. `GET /api/leaderboard?limit=50` is public.
2. `LeaderboardService.top` loads **every** user.
3. For each user it runs one count query: how many `user_progress` rows have `solved = true`.
4. Users with zero solved problems are dropped, the rest sorted by count descending, and the first `limit` returned.

With N users this makes N + 1 queries per page view. That is fine for hundreds of users and slow for tens of thousands. The README and code comments point to the intended upgrade: a Redis sorted set updated on each first accept, read with one command. Users with equal counts appear in an unspecified order; there is no tie-breaker such as "solved first".

PART IV

Specific cases

Concrete situations, what the system does in each, and what the user sees. Every case was worked out from the code; the ones marked "tested" were also reproduced. Case ids are used elsewhere in the handbook.

EVERYONE DEVELOPER

17 Judging cases

A1 A correct C++ solution

ACCEPTED

SITUATION

Problem "Sum of Two" with 1 public and 2 hidden tests. The user submits a correct program.

WHAT HAPPENS

One compile container runs `g++` and exits 0. Three test containers run `./sol`, each printing the right sum. Three ACCEPTED result rows are written, score 100, runtime is the slowest test (typically 400–900 ms, mostly Docker start-up). `user_progress` is created with attempts 1 and solved true.

USER SEES

PENDING for a few seconds, then ACCEPTED, Score 100%, and three rows: Sample 1, Hidden 2, Hidden 3. Their leaderboard count goes up by one. A second accepted submission later does not raise it again.

A2 Passes the samples, fails a hidden test

WRONG_ANSWER

SITUATION

The solution uses `int`, and a hidden test sums two numbers near 2 billion, which overflows.

WHAT HAPPENS

Test 1 accepted, test 2 prints a negative number and is marked WRONG_ANSWER, test 3 still runs and passes. Overall verdict is the first failure: WRONG_ANSWER. Score round(200/3) = 67.

USER SEES

WRONG_ANSWER, 67%, with "Hidden 2" in red. They do not see the hidden input or their output for it; that is deliberate.

A3 Code that does not compile

`COMPILE_ERROR`

SITUATION

A missing semicolon in C++, or a type error in Java.

WHAT HAPPENS

The compile container exits non-zero. The compiler's message (up to 4,000 characters) is saved. No tests run, no result rows, and the attempt is not counted in `user_progress`.

USER SEES

COMPILE_ERROR, 0%, and the compiler output in a red box, e.g. `solution.cpp:5:14: error: expected ';' before 'return'`.

A4 An infinite loop

`TIME_LIMIT_EXCEEDED`

SITUATION

`while (true) {}` on a problem with 10 tests and the default 2,000 ms limit.

WHAT HAPPENS

Each test waits the full 4-second timeout, then the judge kills the local `docker` client process and records TLE. All 10 tests do this, so judging takes about 40 seconds. Killing the client does **not** stop the container: the daemon keeps it running, and since the loop never ends, `--rm` never fires. Ten containers are left spinning, each using up to a full CPU core, until someone runs `docker kill` or restarts Docker.

USER SEES

TIME_LIMIT_EXCEEDED after roughly 40 seconds of PENDING. Over time, every user sees slower judging and eventually spurious TLEs as leaked containers pile up. This is the most serious known issue (chapter 29).

A5 A crash

`RUNTIME_ERROR`

SITUATION

C++ reads past an array (segmentation fault, exit code 139), Python raises an uncaught exception (exit code 1), or Java throws (exit code 1).

WHAT HAPPENS

The run finishes in time with a non-zero exit code: RUNTIME_ERROR. The judge logs the exit code and the first 500 characters of standard error at WARN level on the server.

USER SEES

RUNTIME_ERROR with no explanation. The traceback is in the server log only. Hiding it prevents a solver from printing hidden test data through error messages, but also makes debugging harder.

A6 Using too much memory

`RUNTIME_ERROR`

SITUATION

A program allocates and fills a 1 GB array on a problem whose page says "256 MB".

WHAT HAPPENS

The container's 256 MB cap is hit. In C++ the kernel's out-of-memory killer ends the process with SIGKILL (exit code 137); in Java the JVM throws `OutOfMemoryError` and exits 1; in Python the process is either killed the same way or raises `MemoryError` and exits 1. All of these are non-zero exits: RUNTIME_ERROR. The problem's own `memoryLimitMb` is never used, so a problem set to 64 MB or 1,024 MB is still judged at 256 MB.

USER SEES

RUNTIME_ERROR, never MEMORY_LIMIT_EXCEEDED.

A7 Printing a lot of output

`TIME_LIMIT_EXCEEDED (wrongly)`

SITUATION

A correct solution prints 200,000 numbers (about 1 MB).

WHAT HAPPENS

The judge waits for the process to finish *before* reading its output. Operating-system pipes hold roughly 64 KB; once full, the program blocks waiting for someone to read, and nobody does. The wait times out and the test is marked TLE, even though the program was correct and fast. The container is also leaked, as in A4.

USER SEES

TIME_LIMIT_EXCEEDED on a correct solution. Problems with large outputs cannot currently be judged.

A8 Whitespace differences

`usually ACCEPTED`

SITUATION

The program prints a trailing space after each number, an extra newline at the end, or Windows line endings.

WHAT HAPPENS

Normalization (15.8) removes these differences, so the answer is accepted. One combination slips through: a space immediately before a Windows line ending (`"abc \r\n"`) is not removed, and the answer is judged wrong. This was tested directly.

USER SEES

ACCEPTED in nearly all cases. Programs running in the Linux sandbox print `\n`, so the edge case requires deliberately printing `\r\n`.

A9 Java with a different class name

`COMPILE_ERROR or RUNTIME_ERROR`

SITUATION

The user writes `public class Solution` as on other sites.

WHAT HAPPENS

The file is always saved as `Main.java`. A public class must match its file name, so `javac` fails: COMPILE_ERROR ("class Solution is public, should be declared in a file named Solution.java"). If the class is not public, it compiles to `Solution.class`, but `java -cp . Main` cannot find `Main`: every test is RUNTIME_ERROR.

USER SEES

A compiler message in the first variant; an unexplained RUNTIME_ERROR in the second. The editor's Java template uses `public class Main`, which avoids both.

A10 Reading more input than exists

`RUNTIME_ERROR or WRONG_ANSWER`

SITUATION

A Python solution calls `input()` three times, but the test has two lines.

WHAT HAPPENS

Standard input is closed after the test data, so the third call gets end-of-file and raises `EOFError` (exit 1): RUNTIME_ERROR. In C++, `cin >> x` at end-of-file just fails silently and leaves `x` unchanged, which usually produces WRONG_ANSWER. Nothing hangs, because the input stream is always closed.

USER SEES

RUNTIME_ERROR (Python, Java) or WRONG_ANSWER (C++).

A11 Trying to reach the network

`RUNTIME_ERROR`

SITUATION

A submission tries to download something or send hidden test data to an outside server.

WHAT HAPPENS

`--network=none` gives the container only a loopback interface. DNS lookups and connections fail immediately; an uncaught error exits non-zero.

USER SEES

RUNTIME_ERROR. Nothing leaves the machine.

A12 A fork bomb

`RUNTIME_ERROR or TIME_LIMIT_EXCEEDED`

SITUATION

A program that creates processes endlessly.

WHAT HAPPENS

`--pids-limit=64` makes the 65th process creation fail. Depending on how the program handles that failure, it crashes (RE) or keeps retrying until the timeout (TLE, with a leaked container as in A4). The host is not affected beyond one CPU core.

USER SEES

RUNTIME_ERROR or TIME_LIMIT_EXCEEDED.

A13 Writing a huge file

`varies`

SITUATION

A program writes gigabytes into its working directory.

WHAT HAPPENS

The root file system is read-only and `/tmp` is capped at 32 MB, but `/box` is a normal folder on the host with no size limit. Within the 3–7 second timeout a fast program can write several gigabytes to the host's disk. The folder is deleted when judging ends, so the space comes back; but several such submissions at once could fill the disk and disrupt Postgres.

USER SEES

Probably TIME_LIMIT_EXCEEDED or WRONG_ANSWER. The real effect is on the server.

A14 A slow compile

`COMPILE_ERROR`

SITUATION

Heavy C++ template code takes longer than 20 seconds to compile, or the 256 MB memory cap stops the compiler.

WHAT HAPPENS

A timeout is reported as exit code −1 with the message "timed out". Any non-zero exit is a compile error.

USER SEES

COMPILE_ERROR with "timed out" or a compiler crash message.

A15 A problem with no input

`cannot be created as written`

SITUATION

An admin wants a "print Hello, World" problem with empty input.

WHAT HAPPENS

Test case input is `@NotBlank` on the server and `required` in the form, so an empty input is rejected. The workaround is a dummy input line that the solution ignores. The same rule forbids an empty expected output.

USER SEES

The admin form refuses to submit.

A16 Several submissions at once

`queued`

SITUATION

Twenty people submit in the same second.

WHAT HAPPENS

Each gets its own submission id and workspace folder, so they cannot interfere. The executor has 8 worker threads: 8 submissions are judged in parallel and 12 wait in the queue as PENDING. Each worker holds one of the 10 database connections for its whole judging time, leaving only 2 for all web requests while 8 judges are busy.

USER SEES

Longer PENDING times under load, and potentially slow page loads when judging is saturated.

EVERYONE DEVELOPER

18 Account and access cases

B1 Registering an email that already exists

`409`

WHAT HAPPENS

`existsByEmail` is true; the service throws 409 "Email already registered".

USER SEES

"Email already registered" under the form.

B2 The same email in different capitals

`allowed`

WHAT HAPPENS

"Ada@Example.com" and "ada@example.com" are different strings, so both registrations succeed and create two accounts. Logging in must use the exact capitalisation registered.

USER SEES

Two separate accounts, or a login failure if they type the email differently.

B3 Two users with the same username

`allowed`

WHAT HAPPENS

Username has no unique constraint. Both accounts exist.

USER SEES

Two identical names on the leaderboard with no way to tell them apart.

B4 A username that is too short

`400`

WHAT HAPPENS

The server returns `{"username": "size must be between 3 and 30"}`. The Register page looks for an `error` key, finds none, and falls back to its generic text.

USER SEES

"Registration failed", with no hint about the username rule. (The password minimum is enforced in the browser too, so that one is caught earlier.)

B5 Wrong password or unknown email

`500`

WHAT HAPPENS

The authentication manager throws `BadCredentialsException`, which the catch-all handler turns into a 500.

USER SEES

"Internal server error: Bad credentials". Functionally correct, but it looks like a crash.

B6 Access token expires during use

`transparent`

SITUATION

A user has had a problem open for 20 minutes and presses Submit.

WHAT HAPPENS

The filter ignores the expired token, the endpoint answers 401, the Axios interceptor exchanges the refresh token for a new pair and replays the submit.

USER SEES

Nothing unusual; the submission goes through. If several requests fail at the same moment, each performs its own refresh. That works, because old refresh tokens stay valid.

B7 Returning after more than 7 days

`logged out`

WHAT HAPPENS

The page still shows the username (it comes from `localStorage`). The first protected call returns 401; the refresh call fails because the refresh token has expired; the interceptor clears storage and redirects to `/login`. Public pages (problem list, problem view, leaderboard) keep working, since they do not need a token.

USER SEES

They appear logged in until they do something that needs an account, then they land on the login page.

B8 Promoting a user to admin

`needs a re-login in the UI`

WHAT HAPPENS

An operator runs `UPDATE users SET role='ADMIN' WHERE email='...'`. The backend reads the role from the database on every request, so admin API calls work immediately. The frontend's copy of the role in `localStorage` still says USER.

USER SEES

No admin buttons until they log out and back in.

B9 Demoting or deleting a user who is logged in

`immediate`

WHAT HAPPENS

A demoted admin's next admin call gets 403, because the filter loads the current role. A deleted user's token no longer finds a user, so the request is unauthenticated (401); the refresh also fails ("User not found"), and they are sent to the login page.

B10 Opening someone else's submission

`403`

SITUATION

A user changes the URL from `/submissions/31` to `/submissions/30`.

WHAT HAPPENS

`getById` compares the owner's id with the requester's and throws 403 unless the requester is an admin.

USER SEES

"Not authorized to view this submission". Admins can view any submission.

B11 A normal user tries admin actions

blocked

WHAT HAPPENS

Visiting `/admin/new-problem` redirects to the home page. Calling `POST /api/problems` directly (with curl, or after editing `localStorage` to say ADMIN) is stopped by the URL rule with 403.

USER SEES

A redirect, or "You don't have permission to do that".

B12 Pressing Submit while logged out

code lost

WHAT HAPPENS

ProblemDetail navigates to `/login`. The editor content lives only in that page's state, which is discarded.

USER SEES

After logging in they land on the home page, and the code they typed is gone.

EVERYONE DEVELOPER

19 Problem management cases

C1 Two problems with the same title

works

WHAT HAPPENS

The second gets a numeric suffix, e.g. `two-sum-4821`. The suffix comes from the clock, so in the rare case it repeats an existing slug, the unique constraint rejects the insert with a 500.

C2 A title with no Latin letters

unreachable problem

SITUATION

Title "数组求和" or "😀😀".

WHAT HAPPENS

Slugify removes every character, producing an empty slug. The problem is saved, and its link is `/problems/`, which matches no frontend route. A second such problem gets the slug `-1234`.

USER SEES

The problem appears in the list but its link leads to a blank page.

C3 Editing a problem nobody has attempted

works

WHAT HAPPENS

Fields and tags are replaced. All test cases are deleted and re-inserted with new ids. The slug stays the same even if the title changes.

C4 Editing a problem that has submissions

500

WHAT HAPPENS

The bulk delete of test cases violates the foreign key from `submission_results`. Postgres rejects it, the transaction rolls back (nothing changes, not even the title), and the generic handler returns "Internal server error: could not execute statement ...".

USER SEES

The edit form shows the error. A typo in a test case can no longer be fixed through the UI after the first submission. This is one of the two most important known issues.

C5 Deleting a problem

depends

WHAT HAPPENS

With submissions: 409, "Cannot delete a problem that already has submissions (N). ...". Without: test cases and the problem are removed, and the page returns to the list.

C6 More than 20 problems

partly hidden

WHAT HAPPENS

The list page always requests page 0 with the default size of 20 and has no paging controls. Problem 21 onwards are reachable only by typing their URL.

C7 A problem with no public tests

allowed by the API

WHAT HAPPENS

The admin form insists on at least one public test, but the API does not. A problem created through the API with only hidden tests shows a "Sample tests" heading with nothing under it.

C8 Changing language in the editor

code replaced

WHAT HAPPENS

Switching from C++ to Python replaces the editor contents with the Python template, without asking.

USER SEES

Their code disappears. Switching back does not restore it.

DEVELOPER ADVANCED

20 Infrastructure cases

D1 Runner images were never built

`misleading verdicts`

WHAT HAPPENS

`docker run judge-runner-cpp` finds no local image and tries to pull it from Docker Hub, which fails. `docker` exits with code 125. For C++ and Java this happens in the compile step, so the user gets COMPILE_ERROR with Docker's message ("Unable to find image 'judge-runner-cpp:latest' locally ... pull access denied"). Python has no compile step, so every test becomes RUNTIME_ERROR with nothing shown.

FIX

Build the three images (chapter 25). This is the most likely first-run problem.

D2 Docker is not reachable from the backend

`misleading verdicts`

WHAT HAPPENS

If the socket is not mounted or Docker Desktop is stopped, the `docker` client exits non-zero with "Cannot connect to the Docker daemon". Results look exactly like D1. If the `docker` program is missing altogether (backend run outside the container without Docker installed), `ProcessBuilder` throws and the submission becomes INTERNAL_ERROR with the exception message.

D3 The backend restarts while judging

`stuck forever`

WHAT HAPPENS

The in-memory executor queue and the open transaction are lost. The transaction rolls back, so the submission is still PENDING in the database. Nothing ever picks it up again; there is no recovery scan at start-up. Containers that were running finish on their own (or leak, if looping).

USER SEES

A submission page that polls PENDING forever.

D4 Docker Engine 29 and the backend's Docker client

`unverified risk`

WHAT HAPPENS

The backend image installs Ubuntu 22.04's `docker.io` client package. Docker Engine 29 refuses clients that use an API version older than 1.44. If the Ubuntu package is older than that, every `docker run` from the backend fails with "client version ... is too old", producing the symptoms of D1. This could not be checked because Docker Desktop was not running when the handbook was written.

FIX

Install `docker-ce-cli` from Docker's own apt repository in the backend Dockerfile.

D5 Compiling the backend locally on JDK 23 or newer

`build fails (tested)`

WHAT HAPPENS

From JDK 23, `javac` no longer runs annotation processors found on the classpath unless told to. Lombok never runs, so about 100 "cannot find symbol" errors appear (`getId()` , `builder()` , `log`). The Docker build is unaffected because it uses JDK 17. With processing enabled the project compiles cleanly.

FIX

See 25.3.

D6 No internet connection in the browser

`editor missing`

WHAT HAPPENS

`@monaco-editor/react` downloads the Monaco editor from the jsDelivr CDN when the page first needs it. Offline, or behind a firewall that blocks the CDN, the editor area shows "Loading..." indefinitely.

D7 Many people watching submissions

`load`

WHAT HAPPENS

Each open submission page makes one request every 1.5 seconds until a verdict arrives; each request runs the JWT filter (one user query) and the submission query. A hundred people waiting on a slow queue generate about 67 requests and well over 130 database queries per second. Fine for a demo; it is the reason the README mentions Server-Sent Events.

D8 Postgres data across restarts

`kept`

WHAT HAPPENS

Data lives in the named volume `pgdata` , so `docker compose down` and `up` keep it. `docker compose down -v` deletes the volume and therefore all users, problems and submissions.

PART V

Deep dives

Security, the sandbox, concurrency and the framework details that explain the less obvious behaviour.

ADVANCED

21 Security model

21.1 What is being protected, and from whom

ASSET	THREAT	MAIN CONTROL
The host machine	Malicious submitted code	Docker sandbox (chapter 22)
Hidden test data	Solvers who want to cheat	DTOs without hidden fields; runtime errors not echoed; no network in sandbox
User accounts	Password theft, impersonation	BCrypt hashes; signed short-lived tokens
Problem integrity	Non-admins editing problems	URL rules plus method rules on the server
Other users' code	Reading someone else's submission	Ownership check in `getById`

21.2 Stateless authentication

The server keeps no sessions (`SessionCreationPolicy.STATELESS`). Every request proves its identity with an access token, and the server can verify that token using only the secret key. The benefits are simplicity and horizontal scaling: any number of backend instances can verify any token. The costs are revocation (a token cannot be cancelled before it expires) and the need for a refresh mechanism so short access lifetimes do not force constant logins. This project accepts both costs and does not add a blacklist.

21.3 Why anonymous authentication is disabled

Spring Security normally gives unauthenticated requests an "anonymous" identity. Access to a protected URL by an anonymous identity is treated as *access denied* and, combined with a custom access-denied handler, would come back as 403. The frontend only refreshes tokens on 401. Disabling anonymous authentication (`.anonymous(anon → anon.disable())`) makes a missing or expired token produce "no authentication at all", which goes to the entry point and yields a 401. Public URLs still work because

`permitAll()` grants access without ever looking at the authentication. The comment in `SecurityConfig` documents this reasoning.

21.4 Two layers of authorization

Admin endpoints are protected by URL rules in `SecurityConfig`. Two of them (`getForAdmin` and `delete`) are additionally annotated with `@PreAuthorize`. `create` and `update` rely on the URL rule alone. That is sufficient today, but if someone later moves or renames those routes, the URL rule could stop matching and the endpoints would silently become available to every logged-in user. Adding `@PreAuthorize` to all four admin methods would make protection independent of the URL layout.

21.5 Token storage in the browser

Tokens live in `localStorage`, readable by any JavaScript on the page. If an attacker could inject script (cross-site scripting), they could steal the tokens. The current UI resists that: React escapes all text it renders, problem statements are displayed as plain text (`white-space: pre-wrap`), and there is no `dangerouslySetInnerHTML` anywhere. If problem statements are ever rendered as Markdown or HTML, they must be sanitized, or the tokens should move to `HttpOnly` cookies.

CSRF protection is disabled. That is correct for this design: CSRF attacks rely on the browser attaching credentials automatically (cookies), and this API only accepts credentials from an explicit `Authorization` header.

21.6 Hidden tests and side channels

Hidden test content never appears in a non-admin response, the sandbox has no network, a program's error output is not returned, and each workspace contains only that submission's files (the expected output is never written to disk). A determined cheater can still learn about hidden tests through side channels that every online judge shares: per-test verdicts, deliberate crashes or timeouts that depend on the input, and runtime differences. Defending against that is outside the scope of a project like this.

21.7 Information in error messages

The catch-all handler returns `"Internal server error: " + ex.getMessage()`. For database errors that message can include SQL fragments and constraint names; for type errors, Java class names. It helps during development and gives an attacker a map of the internals in production. A production build should log the detail and return a generic message.

21.8 Secrets and exposed ports

- `JWT_SECRET` is hard-coded in `docker-compose.yml`, and a fallback secret sits in `application.yml`. Anyone who knows the secret can mint an admin token for any email.
- The database password is `cpplatform`.

- Compose publishes Postgres on port 5432 and Redis (no password) on 6379 to all interfaces. On a cloud VM without a firewall, both are reachable from the internet.

All three are fine on a laptop and must change before any public deployment (chapter 28).

21.9 The Docker socket

Mounting `/var/run/docker.sock` into the backend gives the backend full control of the host's Docker engine. Anyone who can run commands as the backend can start a privileged container that mounts the host's root file system, which amounts to root on the host. Submitted code never gets the socket (runner containers do not mount it), so the risk is a vulnerability in the backend itself, not in the sandbox. The README states this trade-off. The standard mitigation is to move judging to a separate machine or VM that holds nothing else of value.

ADVANCED

22 The sandbox, flag by flag

A container is not a virtual machine. It is a normal Linux process that the kernel places inside namespaces (its own view of processes, network, mounts and users) and control groups (resource limits). All containers share the host kernel. The question for a judge is which walls exist and which are missing.

THREAT	CONTROL IN THIS PROJECT	STATUS
Network access, data exfiltration	`--network=none`	`covered`
Memory exhaustion	`--memory=256m --memory-swap=256m`	`covered` (reported as RE)
CPU monopolisation	`--cpus=1`, timeout	`partial`: timed-out containers keep running
Fork bombs	`--pids-limit=64`	`covered`
Tampering with the image	`--read-only`	`covered`
Filling the disk	`/tmp` tmpfs capped at 32 MB	`gap`: `/box` has no quota
Reading other submissions	Only `sub-{id}` is mounted	`covered`
Running as root inside	Runner images use `USER sandboxuser` (uid 10001)	`covered`
Privilege escalation	Docker's default seccomp profile and reduced capability set apply	`default only`: no `--cap-drop=ALL`, no `--security-opt no-new-privileges`
Kernel exploits, container escape	None beyond Docker defaults	`gap`: shared kernel

THREAT	CONTROL IN THIS PROJECT	STATUS
Runaway output	None	`gap`: pipe deadlock (A7)

22.1 The container leak, precisely

On timeout the judge calls `process.destroyForcibly()` on the `docker run` *client* process. The client is only a messenger; the container belongs to the daemon. Killing the messenger does not stop the container, and `--rm` only removes a container after it exits. A robust version names each container (`--name judge-31-4`) and runs `docker kill judge-31-4` on timeout, or wraps the command in the `timeout` utility inside the container so the process ends itself (the class comment mentions a `timeout` backstop, but it is not implemented). Docker's `--stop-timeout` does not help, because nothing ever asks the container to stop.

22.2 The output deadlock, precisely

`process.waitFor(timeout)` is called before anything reads `process.getInputStream()`. The child writes into a pipe of fixed capacity; when it is full, the child's `write` call blocks; the child can then never exit, so `waitFor` times out. The fix is to drain standard output and standard error concurrently, on separate threads or with `ProcessBuilder.redirectOutput` to files, and to cap the amount read so a program cannot exhaust the judge's memory.

22.3 What "time" means here

The measured time is wall-clock time of the `docker run` client, including image lookup, namespace creation, cgroup setup, process start, the program itself, and container removal. On a typical machine the overhead alone is 300–900 ms, varies with load, and is the same for a C++ solution and a Python one. Proper judges measure CPU time of the user process from inside the sandbox (for example from the cgroup's `cpu.stat`, or with `/usr/bin/time`), and keep containers warm instead of creating one per test.

22.4 Stronger isolation options

- **gVisor** (`--runtime=runsc`): intercepts system calls in a user-space kernel, so a kernel exploit has to break gVisor first. Drop-in for this design.
- **Firecracker microVMs**: real virtual machines that start in about 125 ms. Strongest isolation; more infrastructure.
- **isolate** (the sandbox used by the International Olympiad in Informatics): lightweight, measures CPU time and memory precisely, designed for judges.
- Cheap hardening of the current setup: `--cap-drop=ALL`, `--security-opt=no-new-privileges`, `--ulimit fsize=...` to cap file size, and a size-limited tmpfs for `/box` with the files copied in.

ADVANCED

23 Threads and transactions

23.1 Who runs what

THREAD POOL	SIZE	RUNS
Tomcat request threads	up to 200	Every HTTP request, including polling
Spring task executor (`@Async`)	8 core threads, unbounded queue	`judgeAsync`
HikariCP connections	10	Shared by both of the above
OS processes	1 per running `docker` client	Each sandbox run

23.2 Why `submit()` must not be transactional

`submit()` saves the submission and then queues the judge. Because `submit()` has no surrounding transaction, the save commits before the queueing call. If someone later adds `@Transactional` to `submit()`, the commit would move to the end of the method, after `judgeAsync` was queued. The worker thread could then run `findById` before the commit, find nothing, and throw from `orElseThrow()`. The submission would stay PENDING forever, intermittently, depending on timing. The robust pattern is to trigger judging after commit, for example with `@TransactionalEventListener(phase = AFTER_COMMIT)`, or to use a queue (chapter 30).

23.3 One long transaction in the judge

`judgeAsync` is both `@Async` and `@Transactional`. The async proxy moves the call to a worker thread; the transactional proxy opens a transaction on that thread that lasts until judging ends. Consequences:

- **RUNNING is never visible.** Under Postgres's default read-committed isolation, pollers only see committed data.
- **Results appear all at once.** Partial results cannot be shown while judging is in progress.
- **A connection is held for the whole judging time**, up to tens of seconds (case A16).
- **Error handling can fail.** If a database exception occurs inside the loop (say a constraint violation saving a result), Spring marks the transaction rollback-only. The catch block then sets INTERNAL_ERROR and saves, but the commit throws `UnexpectedRollbackException`, everything rolls back, and the submission remains PENDING.

A better shape is short transactions: one to mark RUNNING, one per result (or one at the end), one to finish. The Docker work itself needs no transaction at all.

23.4 How `@Async` works here

`@EnableAsync` makes Spring wrap beans that have `@Async` methods in a proxy. A call from another bean goes through the proxy and is submitted to

the executor; the caller gets control back immediately. Two rules follow. A call from inside `JudgeService` to its own `judgeAsync` would bypass the proxy and run synchronously. And because the method returns `void`, any exception that escapes it is only logged by Spring's default uncaught-exception handler; the caller never learns about it.

23.5 Lazy loading and "open session in view"

Relationships are lazy, so for example `problem.getTags()` issues a query when first called. That only works while a Hibernate session is open. Spring Boot enables `spring.jpa.open-in-view` by default, which keeps a session open for the whole web request, and logs a warning about it at start-up. This is why `ProblemService.list` (not transactional) can map tags, and why `SubmissionService.toView` can read `result.getTestCase().isPublic()`. Inside `judgeAsync` there is no web request; the method's own transaction provides the session.

The `User` principal is loaded inside the security filter, before the open-session interceptor starts, so it arrives in controllers as a detached entity. Using it as a reference when saving a new submission is fine, because only its id is needed for the foreign key.

ADVANCED

24 Spring subtleties

24.1 The JWT filter is registered twice

`JwtAuthFilter` is a `@Component` that implements `Filter`. Spring Boot automatically registers every filter bean with the servlet container, *and* `SecurityConfig` adds it to the security chain. It therefore sits in two chains. It still runs only once per request because it extends `OncePerRequestFilter`, which marks the request after the first pass. The usual way to make this explicit is a `FilterRegistrationBean` with `setEnabled(false)`.

24.2 Errors that become 500

The `@ExceptionHandler(Exception.class)` catch-all runs before Spring's built-in resolvers, so many errors that Spring would normally answer with a 4xx become 500:

REQUEST	EXCEPTION	SHOULD BE
Wrong password	`BadCredentialsException`	401
Expired refresh token	`ExpiredJwtException`	401
Malformed JSON, unknown enum value	`HttpMessageNotReadableException`	400
`/api/submissions/abc`, `?difficulty=SUPER`	`MethodArgumentTypeMismatchException`	400
Unknown URL with a valid token	`NoResourceFoundException`	404

REQUEST	EXCEPTION	SHOULD BE
Wrong HTTP method	`HttpRequestMethodNotSupportedException`	405
Edit after submissions, slug collision	`DataIntegrityViolationException`	409

An unknown URL *without* a token returns 401 rather than 404, because the "anything else" rule requires authentication before Spring even looks for a handler.

24.3 The `-parameters` compiler flag

`@PathVariable Long id` and the JPQL parameter `:problemId` in `TestCaseRepository` are bound by matching Java parameter names. Those names only exist at runtime if the code is compiled with `-parameters`. The Spring Boot parent POM enables it, so Maven builds work. Building with a tool that does not set it fails at runtime with "name for argument … not specified". Explicit names (`@PathVariable("id")` , `@Param("problemId")`) remove the dependency.

24.4 Lombok and booleans named `isX`

For the field `boolean isPublic` , Lombok generates the getter `isPublic()` and the setter `setPublic()` , while the builder method is `isPublic(...)` . The code uses the getter and builder only, so this is consistent. The JSON property in the DTO records is `isPublic` ; with Jackson 2.17 records serialize and deserialize with that exact name (tested).

24.5 Schema management

`ddl-auto: update` adds tables and columns but never removes, renames or retypes them, and it does not run data migrations. Hibernate 6 also generates check constraints for enum columns; adding a new value to an enum such as `Language` later may not update the existing constraint, and inserts with the new value would then fail. A real deployment should switch to Flyway or Liquibase migrations and set `ddl-auto: validate` .

24.6 Smaller details

- Returning a `Page` directly from a controller works but Spring Data 3.3 logs a warning that the JSON structure is not guaranteed to stay stable.
- The Redis starter is on the classpath and configured; it is unused.
- `judge.default-time-limit-seconds` is defined in `application.yml` and read nowhere.
- `ProblemService.getEntityById` carries a comment saying it is package-private; it is public.
- The `DaoAuthenticationProvider` bean is picked up by `AuthenticationConfiguration` , which is how `AuthenticationManager` knows to use BCrypt and the custom user lookup.

PART VI

Operating it

Running, configuring, fixing and deploying the platform.

DEVELOPER

25 Running it

25.1 Everything in Docker (recommended)

```
# from the cp-platform folder, with Docker Desktop running
docker build -t judge-runner-cpp    -f docker/runners/Dockerfile.cpp    docker/runners
docker build -t judge-runner-python -f docker/runners/Dockerfile.python docker/runners
docker build -t judge-runner-java   -f docker/runners/Dockerfile.java   docker/runners
docker compose up --build
```

Open `http://localhost:3000`. The backend is also directly reachable at `http://localhost:8080`.

25.2 Creating the first admin

```
# register through the UI first, then:
docker compose exec postgres psql -U cpplatform -d cpplatform \
  -c "UPDATE users SET role='ADMIN' WHERE email='you@example.com';"
# then log out and log back in so the UI picks up the role (case B8)
```

25.3 Backend and frontend outside Docker

```
# infrastructure only
docker run -d --name pg -e POSTGRES_DB=cpplatform -e POSTGRES_USER=cpplatform \
  -e POSTGRES_PASSWORD=cpplatform -p 5432:5432 postgres:16-alpine
# runner images: as in 25.1

cd backend && mvn spring-boot:run          # http://localhost:8080
cd frontend && npm install && npm run dev   # http://localhost:5173
```

On JDK 23 or newer, `mvn` fails until Lombok's annotation processing is enabled (case D5). Either use JDK 17 or 21, or add this to the `<plugins>` section of `pom.xml`:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <annotationProcessorPaths>
      <path>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <version>${lombok.version}</version>
      </path>
    </annotationProcessorPaths>
  </configuration>
</plugin>
```

Running the backend directly on Windows has not been tested. The workspace path `/tmp/judge` resolves to `C:\tmp\judge`, the permission change is skipped, and Docker Desktop has to accept the Windows path in `-v`; expect to adjust `JUDGE_WORKSPACE`.

25.4 A smoke test from the command line

```
API=http://localhost:8080/api
# 1. register (then promote to ADMIN as in 25.2) and log in
curl -s -X POST $API/auth/register -H 'Content-Type: application/json' \
  -d '{"email":"admin@example.com","username":"admin","password":"password123"}'
TOKEN=$(curl -s -X POST $API/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"admin@example.com","password":"password123"}' | jq -r .accessToken)

# 2. create a problem
curl -s -X POST $API/problems -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"title":"Sum of Two","difficulty":"EASY","statement":"Print a+b.",
       "testCases":[{"input":"3 4","expectedOutput":"7","isPublic":true},
                    {"input":"-5 5","expectedOutput":"0","isPublic":false}]}'

# 3. submit Python and poll
ID=$(curl -s -X POST $API/submissions -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"problemId":1,"language":"PYTHON","sourceCode":"a,b=map(int,input().split())\nprint(a+b)"}' | jq .id)
sleep 5; curl -s $API/submissions/$ID -H "Authorization: Bearer $TOKEN" | jq .status   # "ACCEPTED"
```

DEVELOPER

26 Configuration reference

SETTING	ENV VARIABLE	DEFAULT	USED BY
`spring.datasource.url`	`DB_HOST`, `DB_PORT`, `DB_NAME`	`localhost:5432/cpplatform`	Connection pool
`spring.datasource.username` / `password`	`DB_USER`, `DB_PASSWORD`	`cpplatform` / `cpplatform`	Connection pool
`spring.jpa.hibernate.ddl-auto`		`update`	Schema creation

SETTING	ENV VARIABLE	DEFAULT	USED BY
`spring.data.redis.host` / `port`	`REDIS_HOST`, `REDIS_PORT`	`localhost:6379`	Nothing (unused)
`server.port`		`8080`	Tomcat
`jwt.secret`	`JWT_SECRET`	placeholder string (62 bytes)	Token signing; at least 32 bytes required
`jwt.access-token-expiry-ms`		`900000` (15 min)	Access tokens
`jwt.refresh-token-expiry-ms`		`604800000` (7 days)	Refresh tokens
`judge.workspace-dir`	`JUDGE_WORKSPACE`	`/tmp/judge` (compose: `/tmp/judge-workspace`)	Submission folders
`judge.docker-memory`		`256m`	Every sandbox run
`judge.docker-cpus`		`1`	Every sandbox run
`judge.docker-pids-limit`		`64`	Every sandbox run
`judge.default-time-limit-seconds`		`5`	Nothing (unused)
`logging.level.com.cpplatform`		`INFO`	Application logs

Any Spring property can also be set as an environment variable in upper-case form, for example `JUDGE_DOCKER_MEMORY=512m`.

Fixed in code rather than configuration: the compile timeout (20 s), the run-timeout formula, the `/tmp` tmpfs size (32 MB), compile-error truncation (4,000 characters), the allowed CORS origins, the polling interval (1.5 s, frontend) and the page size (20).

DEVELOPER

27 Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
Every C++/Java submission is COMPILE_ERROR mentioning "Unable to find image"	Runner images not built (D1)	Run the three `docker build` commands
COMPILE_ERROR mentioning "Cannot connect to the Docker daemon"	Socket not mounted or Docker stopped (D2)	Start Docker Desktop; check the compose volume
COMPILE_ERROR mentioning "client version … is too old"	Old Docker client in backend image (D4)	Install `docker-ce-cli` in the backend Dockerfile
Every Python submission is RUNTIME_ERROR, even correct ones	Same causes as above; Python has no compile step to show the message	Check the backend log for the exit code and stderr
C++ compile fails with a permission error writing `sol`	Workspace not writable by uid 10001	Make sure the workspace is on a Linux file system and the permission change

SYMPTOM	LIKELY CAUSE	FIX
		succeeds
COMPILE_ERROR saying `solution.cpp` does not exist	Host and container workspace paths differ	Mount the workspace at the identical path on both sides
Judging gets slower and slower over the day	Leaked containers from timeouts (A4)	`docker ps`, then `docker kill` the judge containers
Submission stays PENDING forever	Backend restarted mid-judge (D3), or rollback-only error (23.3)	Resubmit; check the backend log
`mvn compile`: hundreds of "cannot find symbol" errors	JDK 23+ without Lombok processing (D5)	25.3
Login shows "Internal server error: Bad credentials"	Wrong email or password (B5)	Check the credentials, including email capitalisation
Admin buttons missing after promotion	Role cached in the browser (B8)	Log out and back in
Editing a problem fails with "could not execute statement"	Problem has submissions (C4)	No UI workaround; fix the code (chapter 29)
Code editor stuck on "Loading..."	CDN blocked or offline (D6)	Allow `cdn.jsdelivr.net` or bundle Monaco locally
502 Bad Gateway right after `compose up`	Backend still starting	Wait 10–30 seconds
Backend exits with a database connection error	Postgres not up, wrong `DB_*` variables	Check `docker compose ps` and the variables
Frontend image build is slow or fails after a local `npm install`	No `.dockerignore`, so local `node_modules` is copied into the image	Add `frontend/.dockerignore` containing `node_modules` and `dist`

DEVELOPER ADVANCED

28 Deploying

The simplest real deployment is the same compose file on one Linux VM. The backend needs access to a Docker daemon, which rules out most serverless and managed-container platforms for the backend itself. Before exposing it to the internet:

1. Set a long random `JWT_SECRET` (for example 64 bytes from `openssl rand -base64 64`) and keep it out of version control.
2. Change the Postgres password.
3. Remove the `ports` entries for Postgres and Redis, or bind them to `127.0.0.1`.
4. Put a TLS-terminating proxy (Caddy or nginx with Let's Encrypt) in front of port 3000, and stop publishing 8080.
5. Fix the container leak (A4) and the output deadlock (A7). On a public site, someone will submit an infinite loop within the first day.

6. Replace detailed 500 messages with generic ones (21.7).
7. Back up the `pgdata` volume, or use a managed Postgres.
8. Consider gVisor for the runner containers, and a dedicated VM for judging if anything else of value runs on the same host (21.9).
9. If the frontend is hosted on a different domain (Vercel, Netlify), add that origin to the CORS configuration in `SecurityConfig`.

PART VII

What's next

Every known issue in one place, and the path from a single-server demo to a scalable judge.

DEVELOPER

29 Known issues

SEVERITY	ISSUE	CASE	FIX OUTLINE
High	Timed-out containers keep running	A4	Name containers; `docker kill` on timeout
High	Problems cannot be edited after the first submission	C4	Update test cases in place by id, or soft-delete them, or cascade results
Medium	Large output becomes TLE	A7	Read output while waiting; cap size
Medium	Submissions stuck in PENDING after a restart or rollback-only error	D3	Re-queue PENDING/RUNNING at start-up; short transactions
Medium	No disk quota on `/box`	A13	`--ulimit fsize`, or a sized tmpfs
Medium	Wrong password, bad input and unknown URLs return 500	B5	Specific exception handlers
Medium	Secrets in compose; databases published on all interfaces	21.8	Environment files; remove port mappings
Medium	Local build fails on JDK 23+	D5	Lombok annotation processor path
Low	Problem memory limit ignored; MLE never reported	A6	Pass `memoryLimitMb` to `--memory`; map exit 137 to MLE
Low	Runtime includes container start-up; loose time limits	22.3	Measure CPU time inside the sandbox
Low	RUNNING status never visible	23.3	Commit status changes separately
Low	Empty slug for non-Latin titles	C2	Fall back to `problem-{id}`
Low	Test case weights ignored in scoring	15.9	Weighted score, or remove the field
Low	Problem list shows only 20 problems	C6	Paging controls
Low	Changing language erases code; logging in erases code	C8, B12	Keep code per language; save drafts locally
Low	Usernames not unique; emails case-sensitive	B2, B3	Unique constraint; lowercase emails on input
Low	Validation errors show a generic message on Register	B4	Display field errors
Low	`"abc \r\n"` not normalized	A8	Replace `\r\n` before trimming spaces

SEVERITY	ISSUE	CASE	FIX OUTLINE
Low	Refresh tokens never invalidated; logout is client-only	13.5	Store refresh tokens server-side with rotation
Low	Leaderboard makes one query per user	16	One aggregate query, or Redis
Low	No automated tests	1	Service tests; a judge integration test with Testcontainers
Low	Unused pieces: Redis, `default-time-limit-seconds`, `memoryKb`, `/users/me`, `/submissions/me`	24.6	Use or remove

ADVANCED

30 Roadmap

The README already names the intended direction. Put together, the target architecture looks like this:

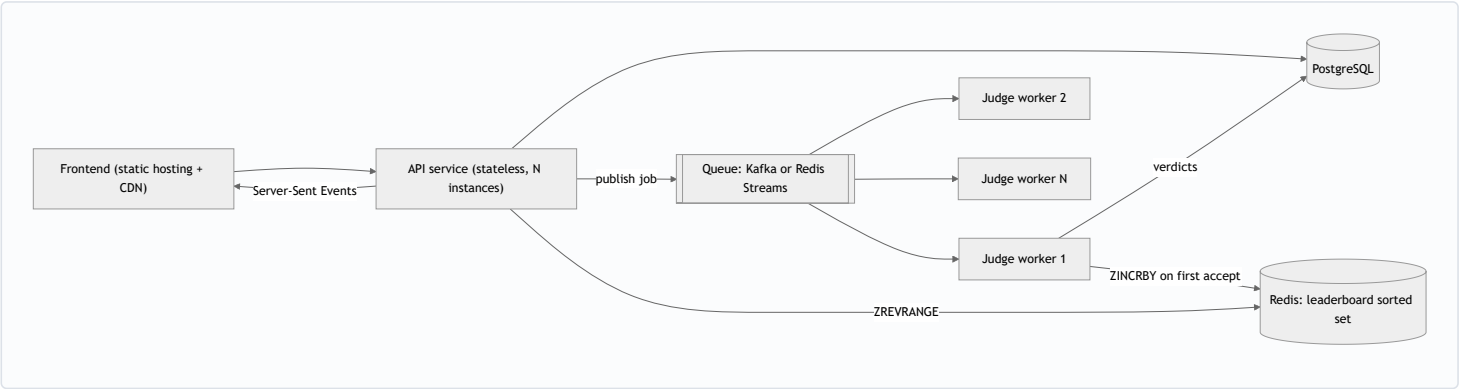


Figure 30.1 · Where the design is heading.

In order of value

1. **Correctness of the judge.** Kill timed-out containers, stream output, enforce memory limits, report MLE, measure CPU time. Without these, nothing built on top is trustworthy.
2. **Fix problem editing.** Edit test cases in place and keep historical results meaningful.
3. **A queue between API and judge.** Submissions become messages; any number of judge workers consume them. This removes the in-memory executor, so restarts no longer lose work, gives back-pressure under load, and lets judging move to separate, isolated machines without the Docker socket on the API host.
4. **Push instead of poll.** Server-Sent Events or WebSockets deliver status changes, including a visible RUNNING state and per-test progress.
5. **Redis leaderboard.** A sorted set keyed by user, incremented on a user's first accept for a problem; reading the top 50 is one command regardless of user count.
6. **Contests.** Time-boxed problem sets, per-contest rankings and penalties; this is where the Redis leaderboard pays off.

7. **Engineering hygiene.** Flyway migrations, tests (service tests plus a Testcontainers judge test), CI, a `.dockerignore`, and specific exception handlers.
8. **Stronger isolation.** gVisor or Firecracker for the runner containers, and dropped capabilities.

Each step can be shipped and measured independently. Step 3 in particular gives a concrete, measurable story: submissions per second before and after adding workers.

CP Platform Handbook · written from a full reading of the cp-platform source (backend Spring Boot 3.3.2, frontend React 18), October 2026. Behaviour marked "tested" was reproduced; everything else was derived from the code.